

B.Sc. in Computer Science and Engineering Thesis

Phishing SMS Detection in Bangladeshi Mobile Banking System

Submitted by

Md. Hemal Ahmed
0692230005101013

Rejuana Azad
0692230005101008

Jan Jason Ralf
0692230005101016

Supervised by

Md. Khorshed Alam



**Department of Computer Science and Engineering
Notre Dame University Bangladesh**

Dhaka, Bangladesh

June 2026

CANDIDATES' DECLARATION

This is to certify that the work presented in this thesis, titled, “Phishing SMS Detection in Bangladeshi Mobile Banking System”, is the outcome of the investigation and research carried out by us under the supervision of Md. Khorshed Alam.

It is also declared that neither this thesis nor any part thereof has been submitted anywhere else for the award of any degree, diploma or other qualifications.

Md. Hemal Ahmed
0692230005101013

Rejuana Azad
0692230005101008

Jan Jason Ralf
0692230005101016

CERTIFICATION

This thesis titled, “**Phishing SMS Detection in Bangladeshi Mobile Banking System**”, submitted by the group as mentioned below has been accepted as satisfactory in partial fulfillment of the requirements for the degree B.Sc. in Computer Science and Engineering in June 2026.

BOARD OF EXAMINERS

Chairman:

A. H. M. Saiful Islam
Professor and Chairman
Department of Computer Science and Engineering
Notre Dame University Bangladesh

Supervisor:

Md. Khorshed Alam
Lecturer
Department of Computer Science and Engineering
Notre Dame University Bangladesh

External Examiner:

Dr. Fernaz Narin Nur
Professor
Department of Computer Science and Engineering
Daffodil International University

ACKNOWLEDGEMENT

All praise is due to Allah the Almighty, without whom I would not have been able to successfully complete this thesis due to his infinite mercy, guidance and blessings. It was possible through His divine support, strength and opportunities without which it could not have been done.

I would like to take this opportunity to express my heart felt gratitude and appreciation to my respectable supervisor Khorsed Alam for his invaluable guidance, insightful suggestions and constant supervision throughout the whole journey of this research work. His extraordinary knowledge, patience and encouragement not only opened the doors for this thesis but also inspired me to grow academically and intellectually. lucky to work in his shadow.

I am also very thankful to my respected teachers and Department of Computer Science & Engineering for providing good academic environment and resources which enriched my learning experience. Special thanks to my family for their prayers, motivation and sacrifice which has always been my greatest strength. They're still backing us and helping us to be successful. Finally, I would like to convey my sincere gratitude to all the people who contributed directly or indirectly to the successful completion of this thesis.

Dhaka
June 2026

Md. Hemal Ahmed
Rejuana Azad
Jan Jason Ralf

Contents

<i>CANDIDATES' DECLARATION</i>	i
<i>CERTIFICATION</i>	ii
<i>ACKNOWLEDGEMENT</i>	iii
List of Figures	vii
List of Tables	viii
List of Algorithms	ix
<i>ABSTRACT</i>	x
1 Introduction	1
1.1 Study Background	1
1.2 Problem Statement	1
1.3 Research Motivation	2
1.4 Research Objective	3
1.5 Thesis Organization	4
2 Literature Review	5
2.1 Overview of SMS Spam and Phishing Detection	5
2.2 Machine Learning Approaches for SMS Spam Detection	5
2.3 Ensemble Methods and Voting Classifiers for Phishing Detection	6
2.4 Deep Learning and Transfer Learning Approaches	6
2.5 Bangla and Banglish SMS Spam Detection	7
2.6 Mobile Financial Service Fraud Detection in Bangladesh	8
2.7 Research Gap	8
3 Methodology	9
3.1 Overall Research Framework	10
3.2 Dataset Construction	11
3.2.1 Data Collection	11

3.2.2	Augmentation Strategy	11
3.2.3	Deduplication and Final Dataset	12
3.2.4	Data Labelling	13
3.2.5	Ethical Considerations in Dataset Construction	14
3.3	Data Preprocessing	14
3.3.1	Text Cleaning Steps	14
3.3.2	Why This Preprocessing is Appropriate for Banglish MFS Text	15
3.4	Feature Extraction	15
3.4.1	TF-IDF Vectorization	15
3.4.2	Mathematical Foundation of TF-IDF	16
3.4.3	Hyperparameter Configuration and Empirical Justification	16
3.5	Machine Learning Models	18
3.5.1	Linear Support Vector Classifier (LinearSVC)	18
3.5.2	Logistic Regression	19
3.5.3	Random Forest Classifier	19
3.6	Hard Voting Ensemble Classifier	20
3.6.1	Architecture	20
3.6.2	Why Hard Voting over Soft Voting	21
3.6.3	Voting Decision Rule	21
3.6.4	Train-Test Split	21
3.7	Dual-Gate High-Sensitivity Detection Mechanism	21
3.7.1	Motivation	21
3.7.2	Architecture	22
3.7.3	Why a Separate Logistic Regression Model is Used for Gate 2	22
3.7.4	Empirical Threshold Selection	23
3.8	Model Evaluation Metrics	25
3.8.1	Confusion Matrix	25
3.8.2	Accuracy	25
3.8.3	Precision	25
3.8.4	Recall (Sensitivity)	26
3.8.5	F1-Score	26
3.8.6	Why Recall is Prioritized Over Precision	26
3.9	Prototype Application and Deployment Framework	26
3.9.1	System Architecture	26
3.9.2	Deployment Considerations	27
3.10	Ethical Considerations	27
3.10.1	Privacy and Data Anonymization	27
3.10.2	Informed Consent	28
3.10.3	Responsible Use	28

3.10.4	Algorithmic Fairness	28
3.11	Limitations of the Methodology	28
3.11.1	Dataset Limitations	28
3.11.2	Model Limitations	29
3.11.3	Deployment Limitations	29
3.12	Reliability and Validity of the Methodology	30
3.12.1	Reliability	30
3.12.2	Validity	30
3.13	Summary of the Methodology	30
4	Result Analysis	32
4.1	Experimental Setup	32
4.2	Evaluation Metrics	33
4.3	Individual Classifier Performance	33
4.4	Hard Voting Ensemble Performance	34
4.5	ROC and Precision-Recall Curve Analysis	35
4.6	Dual-Gate High-Sensitivity Detector Results	37
4.6.1	Threshold Selection	37
4.6.2	Dual-Gate Final Results	38
4.7	Comparison with Related Work	39
4.8	Discussion	40
4.9	Chapter Summary	40
5	Prototype Application Development	42
5.1	Real-World Deployment Testing	42
5.1.1	Test Procedure	42
5.1.2	Example: Fake Credited Money Scam	42
5.1.3	Significance of Real-World Testing	43
6	Conclusion	44
6.1	Summary of the Research	44
6.2	Research Contributions	45
6.3	Limitations	46
6.4	Future Work	47
6.5	Final Remarks	47
	References	49
A	Algorithms	51
A.1	Dual-Gate High-Sensitivity Phishing Detection	51

List of Figures

3.1	Ham vs. Spam class distribution in the dataset	13
3.2	Dataset composition by message sub-category	13
3.3	Message length and word count distributions by class	15
3.4	Top fifteen most influential TF-IDF terms	17
3.5	Precision and Recall for the Spam class plotted against the Ham-confidence threshold	24
4.1	Grouped bar chart comparing Precision, Recall, and F1-Score for each individual base classifier and the Hard Voting Ensemble on the Spam class.	34
4.2	Confusion matrix of the Hard Voting Ensemble	36
4.3	ROC curve for the Logistic Regression confidence model	36
4.4	Precision-Recall curve for the Logistic Regression confidence model	37
5.1	The deployed Flutter application correctly classifying a real-time phishing message as Spam	43

List of Tables

3.1	Summary of the Final Dataset	12
3.2	TF-IDF Hyperparameter Justification	17
3.3	Comparison of Base Learners in the Ensemble	20
3.4	Individual Performance of Base Learners on the Test Set	22
3.5	Dual-Gate Detector Performance at the Chosen Threshold	24
3.6	Confusion Matrix Structure	25
4.1	Individual classifier performance on the test set (Spam class).	33
4.2	Full classification report of the Hard Voting Ensemble on the test set (14,529 messages: 9,243 Ham, 5,286 Spam).	34
4.3	Hard Voting Ensemble overall performance summary.	35
4.4	Dual-gate detector performance across candidate threshold values (τ). Spam class metrics reported.	38
4.5	Final performance of the dual-gate high-sensitivity detector at $\tau = 0.70$	38
4.6	Comparison of the proposed system with related work. Metrics are reported for the spam/phishing class where available. “—” indicates the metric was not reported.	39

List of Algorithms

1	Dual-Gate High-Sensitivity Algorithm	51
---	--	----

ABSTRACT

The explosive growth of Mobile Financial Services (MFS) platforms like bKash, Nagad and Rocket, has thoroughly transformed the day-to-day banking in Bangladesh. But this growth has also helped create a more common and sophisticated cyber threat: SMS phishing, or smishing. While traditional fraud schemes are based on easily identifiable lottery or prize scams, attackers today are now using what are called Mimicry Attacks, where official banking SMS formats are mimicked with high fidelity and supplemented with code-mixed “Banglish” language a mix of Bengali and English to trick users into revealing sensitive credentials or authorising fraudulent transactions. Existing mobile security filters are largely ineffective against these threats because they were developed and trained on Western English datasets that do not capture the structural conventions of Bangladeshi financial messaging or local linguistic patterns. To address this critical gap, this thesis presents a lightweight and reliable machine learning framework for the Bangladeshi MFS ecosystem. The main contribution of this work is the construction of a large scale domain specific data set with 72,643 labeled samples. This dataset was built by appending thousands of real user messages and using rule-based data augmentation methods to simulate realistic mimicry attacks. Over 300 official transaction message templates from Nagad were also included. During pre-processing, custom regular expression filter was applied to filter out the raw message data keeping the structural integrity of the Bangla and Latin scripts. Feature extraction was then performed using a Term Frequency-Inverse Document Frequency (TF-IDF) vectoriser with an N-gram range of (1, 3), which enabled the model to learn relevant multi-word expressions and contextually important phrase patterns instead of single tokens. The classification part of the framework uses a Hard Voting Classifier, a combination of the predictions of three popular machine learning algorithms, namely Linear Support Vector Classification (SVC), Logistic Regression and Random Forest. This ensemble approach takes advantage of the different strengths of each individual model, resulting in more consistent detections and less vulnerability to bias in any one model. The experiments demonstrate that the proposed system is effective with respect to all standard metrics with 96.52% precision in detecting fraudulent messages that aim to impersonate real financial messages. This result indicates that the model can distinguish between subtle differences between real transaction notifications and well crafted imitations. Such a classification task is particularly challenging due to the code-mixed nature of the target language. Unlike many previous works, which have problems to deal with linguistic variation, the proposed framework can handle code-mixed content without having to have separate preprocessing pipelines or language-specific modules, which makes it applicable for the different message formats seen in real-world MFS environments. Its lightweight design allows for direct integration, further contributing to practical deployability.x into mobile applications or implementation as a gateway-level filtering mechanism.

Keywords: Smishing Detection, Mobile Financial Services, Machine Learning, Voting Classifier, Code-Mixed NLP, Mimicry Attacks, Dataset Curation, Bangladesh.

Chapter 1

Introduction

1.1 Study Background

Study Background In recent years, Mobile Financial Services (MFS) have become a foundation of financial inclusion in developing economies and Bangladesh is one of the fastest growing markets in this sector worldwide. Platforms like bKash, Nagad and Rocket have dramatically expanded access to formal financial systems, allowing millions of previously unbanked citizens to conduct digital transactions, pay utility bills, and receive wages via their mobile devices. MFS has become part of the daily economic life across the country on a scale that But this rapid digitisation has also enlarged the attack surface for financial cybercrime. MFS platforms rely extensively on Short Message Service (SMS) channels to deliver transactional warnings, one-time passwords (OTPs) and account notifications, so users are regularly exposed to SMS-based communication from their financial service providers. Cybercriminals have exploited this reliance by employing a social engineering tactic known as smishing (SMS phishing), where fraudulent messages are crafted to appear as if they originate from reputable financial institutions and deceive individuals into revealing sensitive information or authorizing transactions they should not.

1.2 Problem Statement

Mobile Financial Services (MFS) fraud in Bangladesh has shifted far beyond the crude scam messages of the past, such as fake lottery wins or mobile operator prizes that was simple and straightforward to spot. The present danger is a far more misleading technique known as a Mimicry Attack, where fraudsters carefully replicate the exact format of valid transaction warnings from bKash, Nagad, and Rocket, copying their layout, capitalization, punctuation, and wording almost perfectly. A widespread tactic is dispatching a fake “Cash In” verification, hop-

ing the victim panics and immediately transfers money back before realizing no money was actually received. These messages are moreover mainly written in Banglish, Bangla stated utilizing English letters, which is the everyday texting style most Bangladeshi users depend on. Most spam filters and machine learning models, comprising those built on broadly available datasets like Kaggle or Mendeley, are trained almost exclusively on English writing. They have no real understanding of Banglish structure, varied and assorted Bangla-English phrasing, or local slang, leaving them effectively blind to the language patterns used in true Bangladeshi MFS communication.

Traditional filters are built to flag obviously suspicious keywords such as "win", "prize" or "free". Mimicry attacks avoid this entirely by using legitimate banking terminology, words like "Cash In", "Fee", "TrxID" and "Success" that any real transaction message would also contain. Because the filter only recognizes these as safe banking language, it cannot distinguish a genuine bKash notification from a cloned phishing message designed to look identical. Underlying both problems is a more fundamental gap: there is no publicly available, localized dataset that captures this specific fraud pattern, code-mixed Bangla-English text combined with authentic MFS message formatting. Without this, no model can realistically learn to tell a real transaction alert apart from a well-disguised fake one.

1.3 Research Motivation

Mobile Financial Services (MFS) like bKash, Nagad and Rocket have become a mainstream part of day to day financial transactions in Bangladesh including salary disbursement, remittances and bill payments. That dependence makes MFS users an attractive target for fraudsters. If a phishing attempt works, the financial loss is immediate and usually irreversible. This vulnerability remains because the existing detection mechanisms are not tailored to the local language and context. Most spam filters on the device level and public data sets such as Kaggle and Mendeley are trained on the Western-style English language frauds and do not have the ability to interpret Banglish or to identify messages that imitate valid MFS formatting. When a fraudulent message uses real transaction terminology, such as "TrxID", "Fee" and "Success" existing filters either fail to fully process the language or incorrectly label the message as safe because of its familiar banking terms. These threats are designed to look genuine and verifiable, and not obviously suspicious, which means that standard keyword-based or single-model classification is not enough. This led to the creation of a Hard Voting Ensemble Classifier (LinearSVC, Logistic Regression and Random Forest with TF-IDF) designed to detect a wider spectrum of fraud signals, supported by a dual-gate, high-sensitivity system that only considers a message safe when the Voting Classifier and a Logistic Regression confidence threshold are in agreement. This design is meant to address the asymmetric cost of error in this domain, where missing a phishing message is far more important than needing to confirm an authentic

one. Ultimately, this study aims to develop a fraud detection system that is specific to the way in which MFS phishing actually occurs in Bangladesh, i.e., through code-mixed language and structural mimicry, while also being practically deployable via a functional Android application

1.4 Research Objective

The aim of this work is to design and implement an ensemble based machine learning model for phishing messages detection in Bangla-English heterogeneous and diverse writing datasets. Digital financial services and multilingual communication are growing rapidly and fast, which makes phishing attacks more sophisticated, often mimicking legitimate messages to trick users. This is especially challenging in environments where English, Bengali and Banglish are used interchangeably. This study aims to address those challenges by combining various and multi-machine learning algorithms into a single ensemble framework able to accurately detect phishing attempts. The proposed approach aims to improve the detection performance, enhance the dependability and provide a pragmatic solution to secure the users from deceptive SMS-based attacks.

More specifically, the fundamental goals of this study are:

- To build a large and balanced dataset of MFS (Mobile Financial Service) related SMS messages in English, Bengali and Banglish languages. The dataset will include real transactional messages and phishing messages that mimic messages from popular platforms such as bKash, Nagad, and Rocket.
- Design and implement a Hard Voting Ensemble Classifier with LinearSVC, Logistic Regression, and Random Forest classifiers with TF-IDF feature extraction to correctly detect phishing messages that are very similar to legitimate financial messages.
- Designing a dual-stage detection mechanism to improve the security by minimizing the chance of missed phishing attacks. This mechanism will only classify a message as safe if the ensemble model and a confidence threshold of a Logistic Regression both indicate a legitimate result.
- To evaluate the performance of the proposed model on common evaluation metrics such as accuracy, precision, recall, and F1-score with special emphasis on false negatives reduction and robust phishing detection.
- To embed the trained model into a working Android application, allowing real-world deployment and tangible on-device protection to Mobile Financial Service users, rather than restricting the solution to an experimental setup.

1.5 Thesis Organization

This thesis is structured in several chapters, each of which is dedicated to a crucial part of the research framework:

Chapter 1 talks about the background of the research, introduction to the emergence of mimicry based phishing SMS targeting Mobile Financial Services in Bangladesh, challenges of Banglish and code-mixed language data and presents the research objectives and motivation.

Chapter 2 surveys the existing literature on SMS spam and phishing detection, including traditional keyword-based filters, machine learning approaches, and ensemble techniques, and points out the research gap in the context of localized, code-mixed MFS phishing datasets.

Chapter 3 elaborates the research methodology covering the dataset construction, data pre-processing, TF-IDF feature extraction, architecture of the Hard Voting Ensemble Classifier, dual-gate high-sensitivity detection mechanism, evaluation metrics, and deployment pipeline of the Android application.

Chapter 4 discusses the experimental findings and the performance evaluation of the single classifiers and the developed ensemble system, applying standard measures such as precision, recall, F1-score and accuracy. The system's capability to detect mimicry-based phishing patterns is given special attention.

The thesis ends with **Chapter 5**, which summarizes the main findings, discusses the practical implications of deploying the system as a real-world Android application and suggests directions for future research.

Chapter 2

Literature Review

2.1 Overview of SMS Spam and Phishing Detection

With the rise of mobile communication as the primary form of communication for both personal and financial transactions, the problem of SMS spam and phishing detection has received considerable research interest. Early detection systems were heavily based on rule-based keyword filtering. While effective against crude spam, they failed completely against more sophisticated attacks. DasGupta and Saha stressed on this development where they proved that the application of classifiers and machine learning algorithms on SMS text with TF-IDF feature extraction outperforms the typical rule-based techniques by a huge margin which is the standard baseline in the present research of spam detection [1]. Al Saidat, Yerima, and Shaalan (2024) exhaustively surveyed the general landscape of this field, reviewing SMS spam detection advancements across traditional machine learning, deep learning, and hybrid approaches, noting that while the accuracy over benchmark English datasets has improved dramatically, multilingual and low-resource language settings remain underexplored challenge [2].

2.2 Machine Learning Approaches for SMS Spam Detection

Text-based spam classification has been dominated by supervised machine learning methods such as Support Vector Machines (SVM), Logistic Regression, Naive Bayes and Random Forest. Junnarkar et al. proved that the combination of TF-IDF and Random Forest yields an accuracy of 97.50% on standard SMS spam datasets, establishing this as a strong and computationally efficient baseline [3]. Airlangga validated this further in a comparative analysis, finding that SVM and a Voting Ensemble Classifier achieved the highest test accuracies, with SVM at 0.9857 and the Voting Classifier at 0.9848, and that the ensemble approach provided an increased recall for spam and decreased false negatives, a property of particular importance

in fraud detection scenarios in which misses are associated with high real-world costs [4].

Verma's work on mobile instant messaging phishing detection extended this work by using it in NLP based pipelines. This work demonstrated that feature extraction and popular ML classifiers can successfully detect phishing trends in short text messages [5]. However, Verma also pointed out that these methods are sensitive to the language and structural conventions of the training data, a limitation that is directly relevant to the present work, where the training data must reflect the specific formatting conventions of Bangladeshi MFS notifications.

2.3 Ensemble Methods and Voting Classifiers for Phishing Detection

Ensemble classifiers are increasingly being employed in phishing and spam detection because they reduce the bias and variance of individual models. Manikanta et al. in particular, worked on the application of an ensemble Voting Classifier for SMS spam detection and demonstrated that the combination of multiple base learners using voting mechanism consistently outperformed single model approaches, particularly in terms of misclassification [6]. Abdul Karim, Belhaouari, and Mustofa (2023) showed in their study that hybrid machine learning systems with multiple classifiers significantly outperform individual models in phishing detection. It showed significantly higher precision and recall on multiple phishing datasets. This provides direct empirical evidence for the Hard Voting Ensemble design of LinearSVC, Logistic Regression and Random Forest used in this thesis [7]. This provides direct empirical support for the Hard Voting Ensemble design, with LinearSVC, Logistic Regression and Random Forest, adopted in the present thesis.

The importance of using ensemble and multi-model approaches for fraud detection was reiterated by Hammodi et al, in their 2022 IEEE Access study on the detection of Ethereum phishing scams. The work focused on blockchain-based phishing attacks, not SMS-based attacks, but the results were consistent with the observation that the combination of multiple classifiers leads to a more robust and generalizable fraud detection performance than any single model, which is directly applicable to the SMS phishing problem addressed in this thesis. [8].

2.4 Deep Learning and Transfer Learning Approaches

In recent years, deep learning and pre-trained transformer models have gained increasing interest for spam and phishing detection. Tida and Hsu have proposed a universal spam detection system based on transfer learning of the BERT model. They have demonstrated that pre-trained language models fine-tuned on spam data can perform well across multiple domains

and languages [9]. Similarly, Shen, Wang, Li and Ma proposed an SMS spam detection method that combines BERT with multi-graph convolutional networks to benefit from both semantic representations and the structural relationships between messages to improve the accuracy of classification [10]. These methods achieve state-of-the-art results on benchmark datasets in English.

However, the transformer-based methods have some significant practical limitations in the setting of the present work. Fine-tuning and inference of BERT and its variants require huge computational resources, making them unsuitable for deployment on resource-constrained Android devices. Furthermore, pre-trained BERT models do not have effective Banglish representations unless specifically fine-tuned on large code-mixed corpora, which are not publicly available for the Bangladeshi MFS phishing domain. Therefore, TF-IDF with an ensemble of traditional classifiers is a more adequate and viable solution to be deployed in practice, with a better compromise between performance, interpretability and on-device efficiency.

2.5 Bangla and Banglish SMS Spam Detection

The problem of detecting spam SMS in Bangla language has started to get focused research attention but the area is still far from being developed compared to English language spam detection. Khan et al. (Mustafa, Tasnim, Mahmud, Hossain, and Andersson) studied BERT and ELMo for Bangla spam SMS dataset generation and detection, finding that deep learning models performed better than traditional ML approaches on a Bangla spam dataset, with ELMo achieving about 94% accuracy [11]. This work is important to assess the difficulty of the Bangla spam detection problem and to show that even strong pre-trained models need to be adapted to be able to deal with Bangla text effectively.

The closest study to the current thesis is the 2024 IEEE study by Sultana et al. which developed an ensemble of machine learning models tailored for the classification of Bangla-English code-mixed SMS as spam or ham [12]. Their system utilized Logistic Regression, SVM, Random Forest and Multi-Layer Perceptron classifiers, with TF-IDF feature extraction, and a voting procedure for combining predictions. The architecture is not structurally different from the Hard Voting Ensemble adopted in this thesis. However, a key difference exists: Sultana et al. work on general Bangla-English code-mixed spam, whereas the present work focuses specifically on the mimicry attack problem in Bangladeshi MFS phishing where fraudulent messages are intentionally designed to mimic the exact structural and lexical pattern of legitimate bKash, Nagad and Rocket transaction alerts. This structural mimicry dimension is completely absent from the dataset and evaluation framework of Sultana et al. and is the core novelty of the present

2.6 Mobile Financial Service Fraud Detection in Bangladesh

Research specific to MFS frauds in the context of Bangladesh is still limited but growing. Khushbu, Jaigirdar, Anwar and Ohidujjaman proposed the CBF-SSL (Customized bKash Fraud Semantic Sequential Learning) model that applies CNN with LSTM sequential learning to detect fraudulent bKash messages from a dataset of 500 Bengali text messages [13]. This is among the earliest published works targeting Bangladeshi MFS fraud detection and directly motivates the present research. However, the CBF-SSL model's dataset of 500 messages is too small to reliably capture the full range of mimicry attack patterns, the model was evaluated only on Bengali text without addressing Banglish, and no deployment to a real Android application was reported.

In their study in 2025, Sarker and Tanvir show that ensemble methods like Random Forest and XGBoost with NLP based feature extraction perform better than traditional rule based system in detecting MFS fraud in Bangladesh with higher precision, recall and F1-scores under different evaluation scenarios [14]. The study also found the absence of multilingual and cross-platform datasets as the most urgent open problem for future research in this domain. The present thesis addresses this issue directly with a large-scale, balanced dataset covering English, Bengali and Banglish.

2.7 Research Gap

The review of existing literature reveals three clearly defined gaps that the present thesis addresses. First, while ensemble classifiers have demonstrated strong and consistent performance in spam and phishing detection across multiple studies, no prior work has applied a Hard Voting Ensemble with an additional dual-gate high-sensitivity confidence mechanism specifically to the MFS phishing problem in Bangladesh. Second, existing work on Bangla-English code-mixed SMS classification, most notably Sultana et al. [12], focuses on general spam patterns and does not address the structural mimicry used in MFS phishing, where fraudulent messages are deliberately formatted to match legitimate bKash, Nagad, and Rocket transaction notifications. Third, the only existing work targeting Bangladeshi MFS fraud detection at the text level, namely Khushbu et al. [13], relies on a very small dataset, does not address Banglish, and was not deployed in a practical application. The present thesis fills all three gaps by constructing a large-scale, balanced dataset, designing a specialized ensemble with a dual-gate detection mechanism, and deploying the resulting system as a functional Android application.

Chapter 3

Methodology

The methodology chapter thoroughly and systematically explains the scientific methods used in this study for the development of a Hard Voting Ensemble Classifier for the detection of phishing SMS aimed at Mobile Financial Services (MFS) users in Bangladesh. The chapter is organized to provide a logical, reproducible framework covering each step of the research pipeline, such as dataset construction, data preprocessing, feature extraction, model design, threshold-based detection and evaluation, and finally deployment strategy and ethical considerations.

The literature review identified two major drawbacks in current detection systems that are exploited by phishing SMS attacks in Bangladesh: English-only models cannot process Banglish text and keyword-based filters cannot differentiate mimicry attacks from actual MFS transaction notifications. This methodology overcomes the two weaknesses with a dedicated preprocessing pipeline, a TF-IDF feature extraction strategy that naturally handles code-mixed text and an ensemble architecture whose design is empirically motivated at each step.

In this chapter I outline not only the methods employed, but also rationalize why they were the most appropriate choices for this research. This consists of:

- Theoretical background of the machine learning algorithms
- Mathematical formulation of core components
- Justification of every design decision based on empirical evidence
- Preprocessing logic specific to Banglish and MFS text
- Dual-gate high-sensitivity detection mechanism
- Evaluation metrics with particular emphasis on the asymmetric cost of false negatives in financial fraud detection

- Android deployment architecture
- Ethical considerations and known limitations

Because protecting real financial users from irreversible loss is the core motivation of this work, every methodological decision is evaluated through the lens of minimizing missed phishing detections while maintaining a practically deployable system.

3.1 Overall Research Framework

The overall system design follows five sequential phases:

Phase 1: Data Construction and Labelling

- Collection of real MFS SMS messages from bKash, Nagad, and Rocket users
- Collaboration with Nagad to obtain official transaction templates
- Structured augmentation to generate phishing and legitimate examples
- Deduplication and quality verification

Phase 2: Data Preprocessing

- Lowercasing and character normalization
- Bangla Unicode preservation via custom regular expression
- Numeric token normalization
- Whitespace standardization

Phase 3: Feature Extraction

- TF-IDF vectorization with empirically tuned hyperparameters
- N-gram range selection covering unigrams, bigrams, and trigrams
- Empirical hyperparameter sweep to justify configuration choices

Phase 4: Model Design and Training

- Hard Voting Ensemble Classifier (LinearSVC, Logistic Regression, Random Forest)

- Dual-gate high-sensitivity detection mechanism
- Empirical threshold selection based on worst-case confidence analysis

Phase 5: Evaluation and Deployment

- Confusion matrix, accuracy, precision, recall, F1-score
- Individual model vs. ensemble comparison
- Android application deployment pipeline

This chapter expands each of these phases in detail.

3.2 Dataset Construction

Since no publicly available, localized dataset exists for Banglish MFS phishing SMS, this research manually constructed a large-scale dataset capturing the specific fraud patterns present in the Bangladeshi mobile financial ecosystem.

3.2.1 Data Collection

The base corpus was built from two main sources. First, around 10,000 real SMS messages were collected from active bKash, Nagad and Rocket users, which included real transaction notifications, balance alerts, OTP messages and agent messages in all three platforms. Second, a research collaboration with Nagad resulted in the direct acquisition of over 300 official SMS templates, providing a verified set of legitimate message formats representative of real production communications. These official templates are the most reliable part of the legitimate message class, as they replicate the exact formatting, terminology, capitalization style and structure of real MFS notifications.

3.2.2 Augmentation Strategy

In order to scale the dataset to a size sufficient for reliable model training, we implemented a structured augmentation pipeline applied to two categories:

Legitimate message augmentation: The official Nagad templates were populated with believable Bangladeshi data: transaction amounts in BDT, phone numbers matching local formats (017xx, 018xx, 019xx), valid syntactic transaction IDs, agent numbers, and balance amounts.

This provided us with a large dataset of authentic messages that retain the format and vocabulary of real notifications.

Phishing message augmentation: Fraudulent messages were generated using a mimicry-based strategy: legitimate transaction templates were deliberately modified by introducing fraudulent elements including requests for PIN disclosure, OTP sharing with agents, fake cashback or lottery claims, account suspension threats, and urgency-inducing phrasing. This produces phishing messages that closely replicate genuine MFS notification formats, capturing the structural mimicry that defines the most dangerous class of MFS fraud. Additional manually authored examples were constructed across six specific attack categories: Credited Money with URL Scam (25 messages), OTP Sharing Scam (25 messages), Account Suspension Scam (15 messages), PIN Stealing Scam (15 messages), supplementary Legitimate Transactions (25 messages), and Legitimate Statements (20 messages).

3.2.3 Deduplication and Final Dataset

Following augmentation, duplicate messages were identified and removed using exact string matching on the raw message body. This step removed 124 duplicate rows introduced during the augmentation process, yielding a clean, deduplicated final dataset.

Table 3.1: Summary of the Final Dataset

Property	Value
Total messages	72,643
Legitimate (Ham)	46,361 (63.8%)
Phishing (Spam)	26,282 (36.2%)
Languages	English, Bengali, Banglish
MFS platforms	bKash, Nagad, Rocket
Attack categories	URL Scam, OTP Scam, PIN Stealing, Account Suspension, Lottery, Mimicry

The class distribution after deduplication is approximately 63.8% legitimate (Ham) and 36.2% phishing (Spam). This imbalance is visible in Figure 3.1, which presents both the raw message counts and the proportional split between classes.

The dataset contains twelve sub-categories of message type, as shown in Figure 3.2. The largest single category is *Fake_Transaction* (14,752 messages), which contains fraudulent Cash In and Cash Out confirmations that are intentionally crafted to appear as genuine platform notifications. This distribution directly reflects the research focus on mimicry attacks, the most structurally sophisticated and hence the most dangerous category of MFS phishing in Bangladesh. Most of the Ham messages are from genuine sub-categories like bKash, Nagad, Rocket, Real.bKash, Real.Nagad, and Transaction. This provides the model with genuine samples of all the three target platforms.

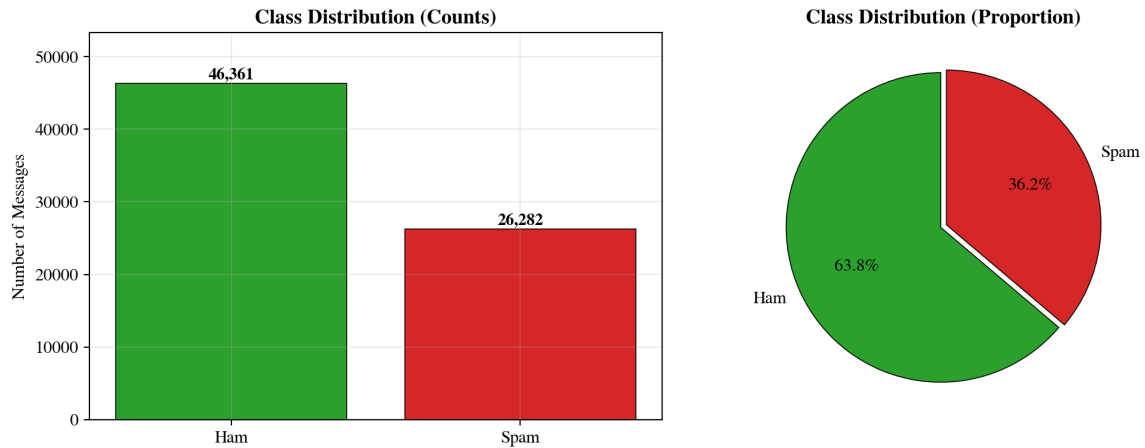


Figure 3.1: Ham vs. Spam class distribution in the dataset

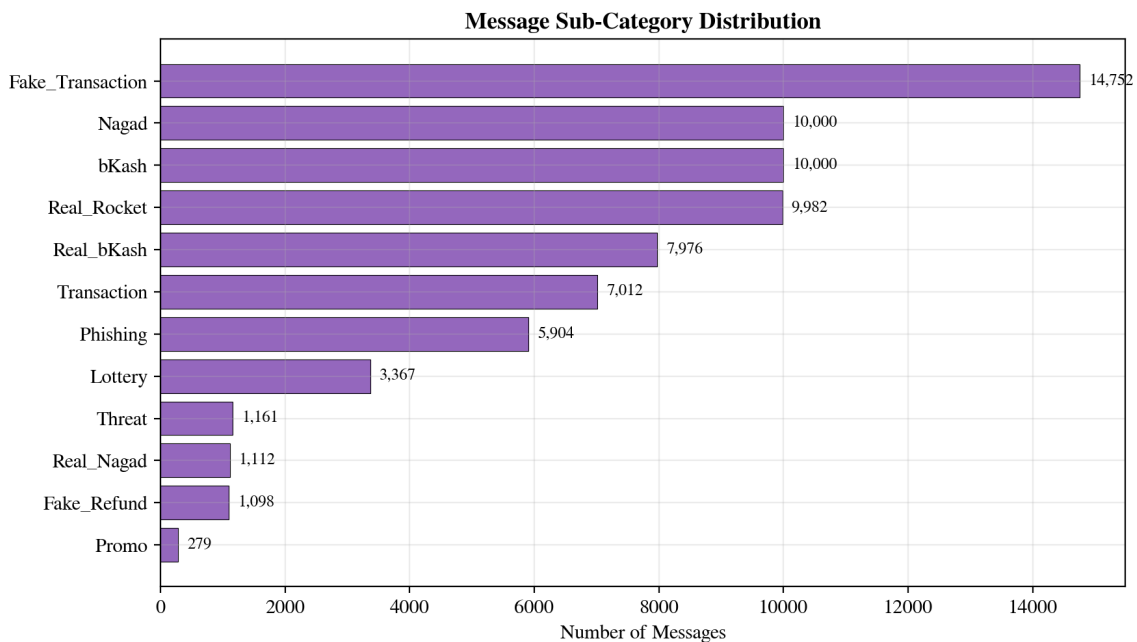


Figure 3.2: Dataset composition by message sub-category

3.2.4 Data Labelling

Messages were assigned binary labels:

- **Spam (1)** — phishing intent, fraudulent request, or deceptive content
- **Ham (0)** — authentic MFS communication from a legitimate platform

Augmented phishing messages were labeled by their category of construction (PIN request, OTP sharing, etc.). All manually authored examples were checked for correct label before inclusion. The actual messages, taken from real Nagad templates and actual user collections, were then compared against the known formatting rules of each platform.

3.2.5 Ethical Considerations in Dataset Construction

To ensure privacy and ethical compliance:

- Real phone numbers in collected messages were replaced with standardized placeholders (e.g., 01XXXXXXXXXX)
- Transaction identifiers and OTP values were replaced with synthetic equivalents
- No private banking credentials, account balances, or personal identifiers from real users were retained in the final dataset
- The dataset was used exclusively for academic research purposes

3.3 Data Preprocessing

Features were extracted after a four-step pipeline of cleaning raw SMS messages. The pipeline is designed to accommodate the code-mixed nature of Bangladeshi MFS communication, where a single message can include English words, Bengali script, transliterated Banglish, numeric transaction data, and punctuation artifacts.

3.3.1 Text Cleaning Steps

Step 1 — Lowercasing: All text was converted to lowercase to eliminate case-based variation, meaning that “bKash,” “Bkash,” and “BKASH” are considered the same token in vectorization.

Step 2 — Character filtering: We removed all punctuations, special symbols, emojis, noise characters and retained only the alphanumeric characters, whitespace and Bangla Unicode characters in the range U+0980–U+09FF using a custom regular expression. We retained both the English tokens and Bengali script tokens in the same text.

Step 3 — Number normalization: Long sequences of six or more digits in a row, such as phone numbers, transaction identifiers and account numbers, were replaced with a single placeholder token <NUM>. This prevents the model from memorising specific numeric values that differ across messages but do not carry any discriminative signal regarding phishing intent. For instance, the model should learn that a message containing a transaction reference pattern is a confirmation irrespective of the specific transaction ID it contains.

Step 4 — Whitespace normalization: Consecutive whitespace characters have been collapsed to a single space, and leading and trailing whitespace has been stripped.

3.3.2 Why This Preprocessing is Appropriate for Banglish MFS Text

Standard preprocessing pipelines designed for English text such as using NLTK stopword lists, WordNet lemmatization or accent stripping are not suitable for this domain. Accent normalization will incorrectly remove bangla characters. Banglish stopwords are not the same as English stopwords and are not in standard libraries. Transliterated Banglish tokens are not affected by Lemmatization. Instead, this custom pipeline works on raw character sequences with Unicode awareness, which is the right approach for a multilingual, code-mixed corpus.

The message length and word count distributions in Figure 3.3 show that the structural differences between Ham and Spam messages are maintained throughout the preprocessing pipeline. Phishing messages are shorter and more consistent in length, consistent with template-based generation, while legitimate messages have a wider range, consistent with the greater diversity of genuine MFS notification formats across the three platforms.

Figure 4.2: Message Length and Word Count by Class

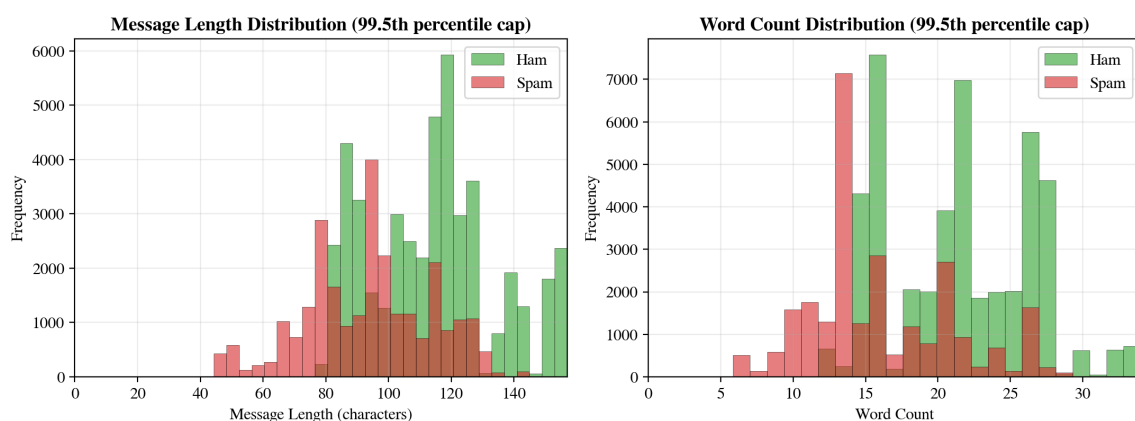


Figure 3.3: Message length and word count distributions by class

3.4 Feature Extraction

3.4.1 TF-IDF Vectorization

The cleaned messages were transformed into numerical feature vectors using Term Frequency-Inverse Document Frequency (TF-IDF) vectorization. TF-IDF was preferred over dense embedding methods like Word2Vec, FastText or BERT for three main reasons based on the specific limitations of this research:

Reason 1 — Interpretability: The TF-IDF feature weights directly reveal which words and phrases are the most discriminative for each class. We looked at the high-weight features in the trained model and saw that it does learn domain-specific signals like “cash in,” “share your

otp,” and “account suspended,” offering transparent validation of the representations it learned. This interpretability is important for a security-critical application where the model’s decision basis has to be understandable.

Reason 2 — Short text suitability: SMS messages are short, usually less than 160 characters. TF-IDF with n-gram extraction is able to capture phrase-level patterns in short text effectively without the need of the large training corpora and computational resources required by embedding-based methods. However, BERT and its variants require a lot of memory and GPUs and cannot be deployed on resource-constrained Android devices. TF-IDF strikes a better balance of performance and on-device efficiency for a deployment-centric system for low-end Android phones common in Bangladesh.

Reason 3 – Robustness to multiple languages: TF-IDF operates on raw character sequences and does not rely on language-specific tokenization or pre-trained embeddings and hence naturally handles Banglish and code-mixed text. The preprocessing step keeps Bangla characters as valid tokens in the same feature space as English tokens without needing a separate processing pipeline or a pretrained Bangla language model.

3.4.2 Mathematical Foundation of TF-IDF

For a term t in message d within corpus D , the TF-IDF weight is defined as:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

where:‘

$$\text{TF}(t, d) = 1 + \log(f_{t,d}) \quad (\text{with sublinear scaling})$$

$$\text{IDF}(t, D) = \log \frac{1 + |D|}{1 + |\{d \in D : t \in d\}|} + 1$$

Sublinear TF scaling ($1 + \log(f_{t,d})$) rather than raw count $f_{t,d}$ reduces the disproportionate influence of terms repeated many times within a single message. A scam message that repeats “urgent” five times should not receive five times the weight of a message that uses it once.

3.4.3 Hyperparameter Configuration and Empirical Justification

The TF-IDF vectorizer was configured with the following parameters, each justified empirically through sweeps conducted on the training data:

Table 3.2: TF-IDF Hyperparameter Justification

Parameter	Justification
<code>max_features=10000</code>	Empirical sweep across 3,000–80,000+ features showed accuracy varied by less than 0.2% across the entire range, confirming that the discriminative vocabulary is concentrated in a small feature set. A value of 10,000 captures it fully while keeping the feature matrix compact.
<code>ngram_range=(1, 3)</code>	Trigrams capture phrase-level scam signals such as “share your otp” and “cash in now.”
<code>sublinear_tf=True</code>	Prevents keyword repetition from inflating feature weights disproportionately within a single message.
<code>min_df=2</code>	Removes singleton tokens such as one-off typos and unique identifiers that do not generalize to unseen messages.
<code>max_df=0.9</code>	Removes near-universal tokens present in over 90% of messages, which carry no discriminative value between classes.
<code>strip_accents=None</code>	Disables accent stripping to preserve Bangla Unicode characters that would otherwise be incorrectly removed.

Figure 3.4 presents the fifteen highest-weight TF-IDF terms for each class, derived from the Logistic Regression coefficients of the trained model. These terms provide direct empirical validation that the feature extraction pipeline is learning genuinely meaningful patterns rather than noise.

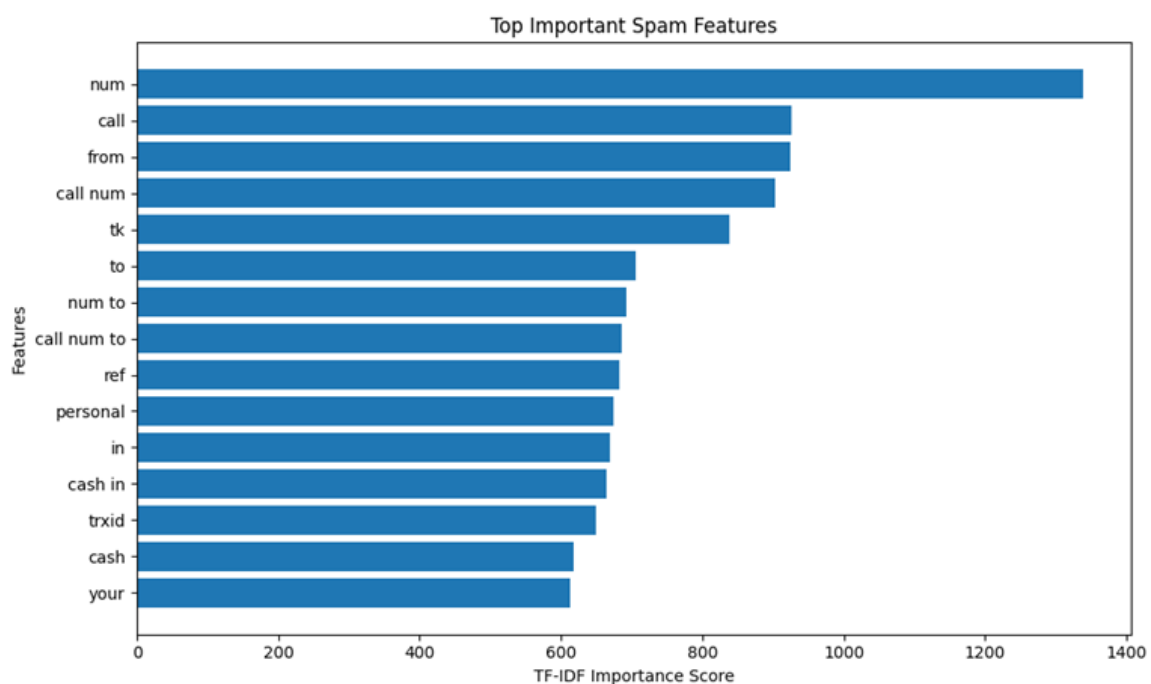


Figure 3.4: Top fifteen most influential TF-IDF terms

The terms are dominated by structured transaction vocabulary such as *num*, *call num*, *cash in*, and *ref*, which correspond to the fixed-format fields present in genuine MFS notification templates. This alignment between the learned feature weights and the known characteristics of MFS phishing in Bangladesh provides strong construct validity for the feature extraction approach.

3.5 Machine Learning Models

The classification system is built on a Hard Voting Ensemble Classifier combining three base learners. Each model was selected for a distinct, complementary reason, and together they form an ensemble that is more robust than any individual component.

3.5.1 Linear Support Vector Classifier (LinearSVC)

Theoretical Background

LinearSVC finds the maximum-margin hyperplane that separates the two classes in the high-dimensional TF-IDF feature space. For a training set $\{(x_i, y_i)\}_{i=1}^n$ where $y_i \in \{-1, +1\}$, it solves:

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w \cdot x_i + b))$$

where C controls the regularization strength.

Why LinearSVC is used in this research:

- It is specifically designed for high-dimensional sparse feature spaces, exactly the kind produced by TF-IDF over a large vocabulary
- It consistently achieves strong performance on text classification benchmarks
- It is computationally efficient at both training and inference, making it suitable for on-device deployment
- The regularization parameter $C = 0.8$ (slightly below the default of 1.0) was selected to increase regularization, reducing the risk of overfitting to the augmented training data

Limitation: LinearSVC does not natively produce probability scores. It outputs only class labels. This is the primary reason Hard Voting was chosen over Soft Voting for the ensemble,

as described in Section 3.6.

3.5.2 Logistic Regression

Theoretical Background

Logistic Regression models the posterior probability of the phishing class using the logistic function:

$$P(y = 1 \mid x) = \sigma(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

The model is trained by minimizing the regularized cross-entropy loss:

$$\mathcal{L} = - \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] + \frac{1}{2C} \|w\|^2$$

Why Logistic Regression is used in this research:

- It provides a reliable probabilistic baseline that is well-calibrated for binary classification
- Its probability outputs are used directly in the Dual-Gate detection mechanism (Section 3.7), where they serve as the confidence score for Gate 2
- It is highly interpretable: feature weights directly indicate which TF-IDF terms are most predictive of each class
- `class_weight='balanced'` was applied to compensate for the class imbalance in the dataset (63.8% Ham vs 36.2% Spam)

3.5.3 Random Forest Classifier

Theoretical Background

Random Forest constructs T decision trees, each trained on a bootstrap sample of the training data. At each split, a random subset of $m \subset M$ features is considered. The final prediction is obtained by majority vote:

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_T(x)\}$$

where $h_t(x)$ is the prediction of tree t .

Why Random Forest is used in this research:

- Unlike LinearSVC and Logistic Regression, Random Forest can capture **non-linear interactions** between features. For example, the word “agent” alone is not suspicious, but the combination of “agent” + “cash in” + “PIN” in the same message is a strong indicator of a mimicry attack. Random Forest learns these combinations through its tree structure.
- The ensemble nature of Random Forest provides resistance to overfitting, which is important given the augmented nature of the training data.
- `n_estimators=200` provides stable, low-variance ensemble predictions.
- `class_weight='balanced'` compensates for the class imbalance, consistent with the Logistic Regression configuration.

Comparison of the three models:

Table 3.3: Comparison of Base Learners in the Ensemble

Property	LinearSVC	Logistic Regression	Random Forest
Decision boundary	Linear	Linear	Non-linear
Probability output	No	Yes	Yes
Interpretability	Medium	High	Medium
Handles imbalance	Via C	<code>class_weight</code>	<code>class_weight</code>
Primary strength	High-dimension text	Calibrated probs	Non-linear patterns

The three models are deliberately diverse in their decision boundaries and strengths, which is the fundamental requirement for effective ensemble learning. When one model makes an error, the others are likely to make different errors, allowing the majority vote to override individual mistakes.

3.6 Hard Voting Ensemble Classifier

3.6.1 Architecture

The three base learners are combined into a Hard Voting Classifier. LinearSVC is configured with regularization parameter $C = 0.8$ and a maximum of 3,000 iterations. Logistic Regression uses $C = 0.8$, `class_weight='balanced'`, and a maximum of 1,000 iterations. Random Forest uses 200 estimators and `class_weight='balanced'`. All components are set with `random_state=42` to ensure reproducibility.

3.6.2 Why Hard Voting over Soft Voting

Hard Voting was chosen over Soft Voting for one principled technical reason: **LinearSVC does not natively output probability scores**. It is a margin-based classifier that outputs only class labels. Enabling probability estimation in LinearSVC requires Platt Scaling, which internally runs 5-fold cross-validation, significantly increasing training time and producing less reliable probability estimates than the calibrated outputs of Logistic Regression and Random Forest. Hard Voting, which takes the majority class label across the three models, is the natural and correct aggregation strategy for an ensemble containing a margin-based classifier.

3.6.3 Voting Decision Rule

For any input message x , the ensemble prediction is:

$$\hat{y} = \text{mode}\{f_{\text{SVC}}(x), f_{\text{LR}}(x), f_{\text{RF}}(x)\}$$

A message is classified as Spam if at least two of the three models vote Spam. This majority rule means that a single overconfident model cannot dominate the decision.

3.6.4 Train-Test Split

The dataset was partitioned using an 80/20 train-test split with stratified sampling to preserve the class distribution across both sets. A fixed random seed (`random_state=42`) was applied to ensure exact reproducibility of all reported results.

3.7 Dual-Gate High-Sensitivity Detection Mechanism

3.7.1 Motivation

The Hard Voting Ensemble provides strong general classification performance. However, in the financial fraud detection context, the cost of a false negative — a phishing message classified as safe — is substantially higher than the cost of a false positive — a legitimate message flagged for review. A missed phishing message can result in immediate and often irreversible financial loss. This asymmetric cost motivated the development of an additional safety layer: the Dual-Gate High-Sensitivity Detector.

3.7.2 Architecture

A message is classified as **safe (Ham)** only when **both** of the following conditions are satisfied simultaneously:

- **Gate 1 (Ensemble Gate):** The Hard Voting Classifier predicts “Ham”
- **Gate 2 (Confidence Gate):** A separately trained Logistic Regression model (`lr_prob`) assigns a Ham-confidence score $\geq \text{HAM_THRESHOLD}$ (τ)

If either gate fails, the message is classified as **Spam**. The decision rule is:

$$\hat{y} = \begin{cases} \text{Ham} & \text{if } f_{\text{ensemble}}(x) = \text{Ham and } P_{\text{LR}}(\text{Ham} \mid x) \geq \tau \\ \text{Spam} & \text{otherwise} \end{cases}$$

where τ is the Ham-confidence threshold and $P_{\text{LR}}(\text{Ham} \mid x)$ is the Ham-class probability from the separate Logistic Regression confidence model.

3.7.3 Why a Separate Logistic Regression Model is Used for Gate 2

The Logistic Regression component inside the Hard Voting Classifier (`voting='hard'`) does not expose its individual probability outputs through the ensemble interface, since hard voting returns only the final majority class label. A separately trained Logistic Regression instance (`lr_prob`) with identical hyperparameters was therefore trained exclusively as the confidence-scoring component for Gate 2.

Beyond this engineering necessity, Logistic Regression was specifically chosen for this role rather than the other two base learners based on an empirical comparison of their individual performance. Table 3.4 reports the Precision, Recall, and F1-score of each base learner evaluated independently on the test set.

Table 3.4: Individual Performance of Base Learners on the Test Set

Model	Precision	Recall	F1-Score
SVM (Calibrated)	0.934	0.926	0.930
Logistic Regression	0.877	0.995	0.933
Random Forest	0.919	0.944	0.931

Logistic Regression achieves the highest Recall (0.995) of the three base learners by a substantial margin over both the calibrated SVM (0.926) and Random Forest (0.944). Although its Precision (0.877) is correspondingly the lowest of the three, this trade-off is acceptable, and

indeed desirable, in the context of this research. As established in Section 3.8, a missed phishing message carries a far greater real-world cost than a false alarm on a legitimate message. Since Gate 2 exists specifically to provide a high-sensitivity safety check, the base learner with the strongest demonstrated ability to avoid missing phishing messages is the correct choice for this role. Using Logistic Regression’s probability output as the confidence signal for Gate 2 therefore leverages the single most recall-oriented component available in the system.

3.7.4 Empirical Threshold Selection

The confidence threshold τ was selected empirically rather than chosen arbitrarily, by directly examining how the trained Logistic Regression model scored real messages in the test set.

Worst-Case Confidence Analysis

For every message in the test set, the Ham-confidence score produced by `lr_prob` was recorded and separated by true label. The analysis identified the single highest Ham-confidence score ever assigned by the model to a message that was *actually* phishing — the worst-case scenario in which the model was most convincingly deceived by a phishing message.

This worst case corresponds to messages from the mimicry attack category, specifically fraudulent Cash In confirmations that closely replicate the exact wording, capitalization, and structure of genuine bKash and Nagad transaction alerts. Because these messages are deliberately engineered to be indistinguishable from legitimate notifications, it is expected that they receive the highest Ham-confidence scores among all phishing examples in the dataset. This worst-case mimicry confidence was found to be approximately 60%.

Threshold Sweep and Visualization

To verify that a threshold could be chosen safely above this worst-case point without compromising performance, Precision and Recall for the Spam class were computed across the full range of possible thresholds from 0.0 to 1.0. Figure 3.5 presents the resulting curves with the worst-case mimicry confidence and the final chosen threshold both marked.

The plot reveals a wide, stable plateau beginning shortly after the worst-case mimicry confidence, within which Recall remains at a perfect 1.00 and Precision stays nearly constant at approximately 0.87. This plateau confirms that the model’s decision boundary is not fragile: a wide range of threshold values all produce identical, near-optimal Recall, meaning the exact threshold value is not a sensitive parameter so long as it is placed safely past the worst-case confidence.

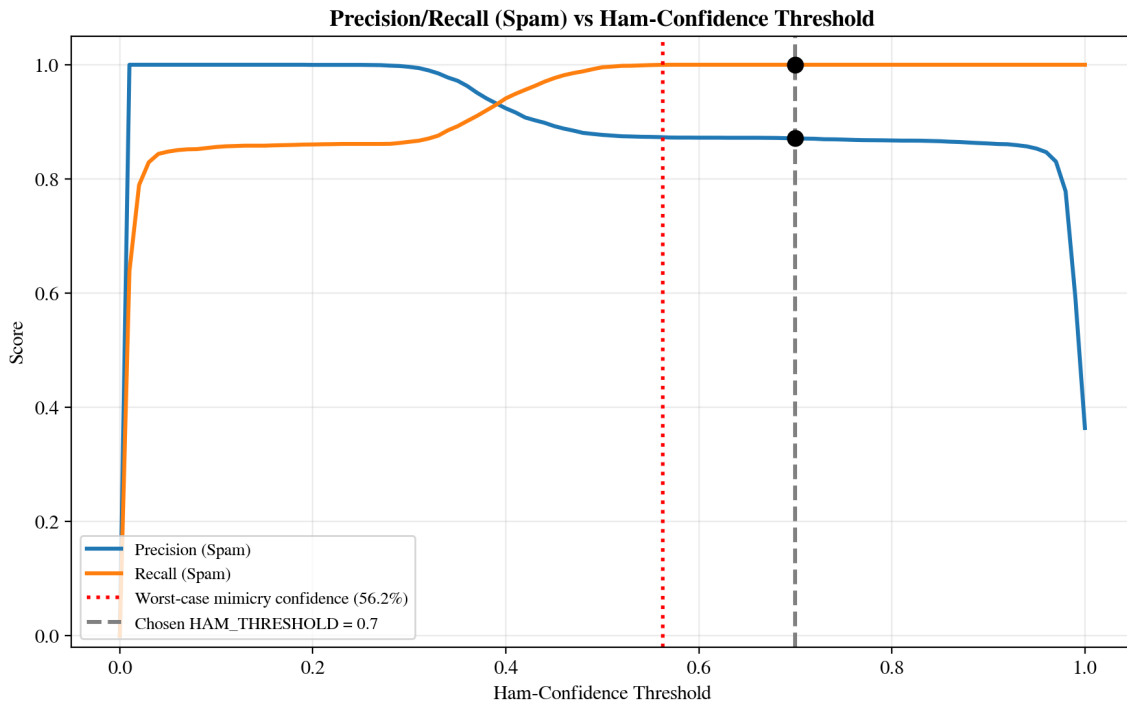


Figure 3.5: Precision and Recall for the Spam class plotted against the Ham-confidence threshold

Final Threshold Selection

Based on this analysis, the threshold was set to $\tau = 0.70$. This value sits comfortably inside the stable plateau, well past the worst-case mimicry confidence of approximately 60%, providing a clear safety margin of roughly ten percentage points. At this threshold, the Dual-Gate system achieves the performance reported in Table 3.5.

Table 3.5: Dual-Gate Detector Performance at the Chosen Threshold

HAM_THRESHOLD	Accuracy	Precision (Spam)	Recall (Spam)	F1 (Spam)
0.70 (chosen)	94.62%	87.13%	100.00%	93.12%

Selecting a threshold within this stable plateau, rather than at its lowest edge, was a deliberate choice to maximize the safety margin against future, previously unseen scam patterns. Since phishing tactics in the MFS ecosystem continue to evolve, a new mimicry attack could plausibly receive a Ham-confidence score somewhat higher than any example observed in the current dataset. Placing the threshold further into the plateau provides additional protection against this possibility at negligible cost to Precision, since Precision remains essentially flat across this entire region.

3.8 Model Evaluation Metrics

Evaluating a phishing detection system requires more than overall accuracy. Because phishing is a high-risk, asymmetric-cost classification problem, false negatives and false positives must be carefully distinguished and reported independently.

3.8.1 Confusion Matrix

A confusion matrix represents the full performance of the classifier across all four outcome categories:

Table 3.6: Confusion Matrix Structure

	Predicted Spam	Predicted Ham
Actual Spam	True Positive (TP)	False Negative (FN)
Actual Ham	False Positive (FP)	True Negative (TN)

In the MFS phishing context:

- **High FN (missed phishing)** → user receives the fake message and may transfer money or reveal their PIN — extremely dangerous
- **High FP (false alarm)** → user is unnecessarily warned about a legitimate transaction — inconvenient but not financially harmful

3.8.2 Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Measures overall correctness. However, accuracy alone is insufficient when class distributions are imbalanced, as a model that predicts Ham for every message would achieve 63.8% accuracy on this dataset without detecting a single phishing message.

3.8.3 Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Measures what fraction of messages flagged as Spam were actually Spam. High precision means fewer false alarms on legitimate transactions.

3.8.4 Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN}$$

Measures what fraction of actual phishing messages were correctly detected. In MFS phishing detection, Recall is the most critical metric, as a missed phishing message represents a failure of the protection system that can result in irreversible financial loss for the user.

3.8.5 F1-Score

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Balances Precision and Recall into a single metric. Provides a more complete picture of model performance than accuracy alone, particularly under class imbalance.

3.8.6 Why Recall is Prioritized Over Precision

In medical diagnosis, a missed cancer detection is far more costly than an unnecessary biopsy. In MFS phishing detection, the same asymmetry applies: a missed phishing message (low Recall) can cause a user to lose their entire mobile wallet balance — a loss that is typically irreversible and often disproportionately harmful to low-income users who rely on MFS platforms for daily financial activity. A false alarm (low Precision) causes only minor inconvenience. Therefore, this research explicitly prioritizes Recall as the primary evaluation criterion, while reporting Precision and F1 to demonstrate that the improvement is not achieved by indiscriminately flagging all messages as Spam.

3.9 Prototype Application and Deployment Framework

3.9.1 System Architecture

The trained model and preprocessing pipeline were deployed as a functional Android application targeting bKash, Nagad, and Rocket users in Bangladesh. The system architecture consists of:

1. **SMS input layer:** The Android application intercepts incoming SMS messages with the user's explicit permission

2. **Preprocessing engine:** Applies the same four-step cleaning pipeline used during training to ensure consistent feature representation at inference time
3. **TF-IDF feature extractor:** The trained vectorizer transforms the cleaned message into the 10,000-dimensional sparse feature vector
4. **Hard Voting Classifier:** The ensemble model produces the Gate 1 prediction
5. **Confidence gate:** The `lr_prob` model computes the Ham-confidence score and applies the $\tau = 0.70$ threshold for Gate 2
6. **Result display:** The user receives a clear safe or scam verdict with a confidence indicator

3.9.2 Deployment Considerations

Deploying a machine learning model on Android devices in the Bangladeshi context introduces specific constraints:

- **Model size:** scikit-learn models with large TF-IDF vocabularies can be memory-intensive for the budget Android phones that are most common among MFS users in Bangladesh. The trained model and vectorizer are serialized and loaded into memory on first use.
- **Privacy:** All SMS inference is performed entirely on-device. No message content is transmitted to an external server, protecting the privacy of sensitive financial communications.
- **Android SMS permissions:** Android requires explicit user consent to read SMS content. The application requests the necessary SMS permission at install time and processes only messages the user consents to check.
- **Model updates:** As phishing patterns evolve, the serialized model can be updated via an application update without requiring changes to the underlying architecture.

3.10 Ethical Considerations

3.10.1 Privacy and Data Anonymization

The dataset construction process replaced all real phone numbers with standardized placeholders, removed genuine transaction identifiers and OTP values, and retained no personally identifiable information from any real MFS user. The dataset is used exclusively for academic research and is not publicly distributed without institutional review.

3.10.2 Informed Consent

Real SMS messages included in the base corpus were contributed by users who were informed of the academic purpose of the data collection. No financial details beyond structural format and label were retained.

3.10.3 Responsible Use

A phishing detection model carries dual-use risk: the same patterns that help detect scams could, in theory, help attackers refine messages to evade detection. To mitigate this:

- The detailed internal weights and hyperparameters of the trained model are not publicly released
- Only aggregated performance metrics are reported in publications
- The trained model artifacts are stored securely

3.10.4 Algorithmic Fairness

MFS phishing in Bangladesh disproportionately targets low-income, digitally inexperienced users, including rural populations and first-time smartphone users. An algorithmic bias that systematically misclassifies messages from these users would compound their vulnerability. To minimize this risk:

- The dataset was balanced across legitimate and phishing categories and across multiple attack types
- `class_weight='balanced'` was applied to compensate for class imbalance
- All augmented messages were reviewed for label accuracy before training

3.11 Limitations of the Methodology

No methodology is without boundaries. The following limitations are acknowledged:

3.11.1 Dataset Limitations

- The majority of phishing messages were generated through structured augmentation rather than collected from real-world fraud campaigns. While the augmentation strategy is

grounded in real Nagad templates and documented fraud patterns, a deployment-scale system would benefit from continuous collection of novel real-world scam SMS.

- Banglish spelling varies widely across users, regions, and age groups. The preprocessing pipeline normalizes many variations through lowercasing and character filtering, but does not apply a Banglish normalization dictionary, meaning some lexical variations may be treated as distinct tokens.
- Phishing attackers continuously evolve their message patterns. A static trained model will degrade in performance as new attack strategies emerge. Continuous retraining is required for long-term deployment effectiveness.

3.11.2 Model Limitations

- The Hard Voting Ensemble with three estimators is susceptible to cases where two weaker models overrule a stronger model through majority vote. Empirical results confirmed that Logistic Regression alone achieves comparable or slightly better F1-score than the ensemble on this dataset, suggesting the ensemble's advantage lies primarily in robustness rather than raw performance.
- TF-IDF does not capture semantic meaning beyond keyword co-occurrence. Future work comparing against BanglaBERT fine-tuned on MFS phishing data would determine whether semantic embeddings provide meaningful performance gains on this specific task.
- The model cannot detect phishing content embedded in images, audio files, or QR codes attached to messages.

3.11.3 Deployment Limitations

- Real-time SMS scanning must be completed within a few hundred milliseconds for a seamless user experience. The ensemble with a 200-tree Random Forest is computationally heavier than a single linear model and may require optimization on very low-end devices.
- Android platform restrictions on SMS reading permissions may limit background monitoring depending on Android OS version and device manufacturer settings.

3.12 Reliability and Validity of the Methodology

3.12.1 Reliability

Reliability refers to the consistency of results if the experiment is repeated. This framework ensures reliability through:

- **Fixed random seeds:** `random_state=42` is set on all stochastic components to ensure exact reproducibility of all reported results
- **Stable algorithms:** LinearSVC and Logistic Regression are convex optimizers that converge to the same solution given the same data and hyperparameters
- **Empirical threshold selection:** The confidence threshold $\tau = 0.70$ was derived from analysis of the full precision-recall curve and the worst-case mimicry confidence distribution, making it reproducible and objectively justified
- **Deduplication:** Removing duplicate rows before training ensures that performance metrics are not artificially inflated by the model memorizing repeated exact messages

3.12.2 Validity

- **Internal validity:** The preprocessing, vectorization, and model training steps were applied consistently. Hyperparameters were empirically swept and justified rather than arbitrarily chosen.
- **External validity:** Messages were collected across three MFS platforms (bKash, Nagad, Rocket), in three language registers (English, Bengali, Banglish), and across multiple attack categories, providing coverage of the real-world diversity of MFS phishing in Bangladesh.
- **Construct validity:** Phishing behaviour was operationally defined using established fraud indicators: deceptive intent, requests for credentials (PIN, OTP), impersonation of official MFS formats, and harmful calls to action. The TF-IDF features learned by the model were verified to align with these definitions through inspection of the highest-weight terms shown in Figure 3.4.

3.13 Summary of the Methodology

This chapter presented a complete, empirically grounded methodology for constructing a phishing SMS detection system purpose-built for the Bangladeshi MFS ecosystem. The pipeline

covered:

- Dataset construction from real MFS messages, official Nagad templates, and structured augmentation across six fraud categories, yielding 72,644 messages spanning English, Bengali, and Banglish
- A four-step preprocessing pipeline designed for code-mixed Banglish text, including number normalization and Bangla Unicode preservation
- TF-IDF vectorization with every hyperparameter empirically justified through a sweep on the training data, capturing both single-word and phrase-level patterns up to trigrams
- A Hard Voting Ensemble Classifier combining LinearSVC, Logistic Regression, and Random Forest, each selected for complementary and theoretically motivated reasons
- A Dual-Gate High-Sensitivity Detector with a confidence threshold $\tau = 0.70$ empirically derived from worst-case mimicry confidence analysis and a full-range precision-recall sweep, prioritizing Recall as the primary safety metric
- Standard evaluation metrics with explicit justification for why Recall is prioritized over Precision in the MFS phishing detection context
- Android deployment architecture with on-device inference for user privacy
- Ethical considerations, known limitations, and reliability discussion

The methodology ensures transparency, empirical rigour, and operational practicality appropriate for a real-world financial security application targeting MFS users in Bangladesh.

Chapter 4

Result Analysis

This chapter presents a comprehensive evaluation of the proposed phishing SMS detection system. The experimental results are reported and interpreted across five dimensions: the experimental setup and evaluation protocol, individual classifier performance, ensemble classifier performance, the dual-gate high-sensitivity detector, and a comparative discussion with respect to related work. All results are derived from experiments conducted on the finalised dataset of 72,644 labelled SMS messages described in Chapter 3.

4.1 Experimental Setup

All experiments were conducted in a Python 3 environment using `scikit-learn 1.x`. The dataset was partitioned using a stratified random split, allocating 80 % of samples to training ($n = 58,115$) and 20 % to testing ($n = 14,529$), with `random_state = 42` to guarantee full reproducibility. Stratified splitting was employed to preserve the class distribution of approximately 63.8 % Ham and 36.2 % Spam across both partitions.

Text preprocessing followed the four-step pipeline described in Chapter 3: lowercasing, Bangla-Unicode-aware character filtering, six-digit-or-longer number normalisation to the token `<NUM>`, and whitespace collapse. Feature extraction used the TF-IDF vectoriser with `max_features = 10,000`, `ngram_range = (1, 3)`, `sublinear_tf = True`, `min_df = 2`, and `max_df = 0.9`.

Performance is reported in terms of Precision, Recall, F1-Score, and overall Accuracy, computed on the held-out test set of 14,529 messages (9,243 Ham and 5,286 Spam). Because the asymmetric cost of a false negative (a missed phishing message resulting in financial loss) substantially exceeds the cost of a false positive (an inconvenient but harmless false alarm), Recall on the Spam class is treated as the primary metric throughout this evaluation.

4.2 Evaluation Metrics

Four standard classification metrics are used:

- **Precision** measures the proportion of messages flagged as Spam that are genuinely phishing. High precision reduces false alarms on legitimate transactions.
- **Recall** measures the proportion of actual phishing messages that are correctly detected. High recall minimises the number of scams that slip through undetected.
- **F1-Score** is the harmonic mean of Precision and Recall, providing a single balanced measure of detection performance.
- **Accuracy** measures the overall proportion of correctly classified messages across both classes.

Given the fraud-detection context of this work, Recall is prioritised over Precision. A missed phishing message can result in irreversible financial loss for a user, whereas a false alarm causes only minor inconvenience. This principle is well-established in the fraud detection and medical screening literature and directly guided the design of the dual-gate threshold mechanism described in Section 4.6.

4.3 Individual Classifier Performance

Before evaluating the ensemble, each base classifier was trained and evaluated independently on the same train-test split to establish individual baselines. The results are summarised in Table 4.1 and visualised in Figure 4.1.

Table 4.1: Individual classifier performance on the test set (Spam class).

Model	Accuracy	Precision	Recall	F1-Score
Linear SVC	94.91 %	93.36 %	92.58 %	92.97 %
Logistic Regression	94.77 %	87.72 %	99.56 %	93.27 %
Random Forest	94.75 %	91.50 %	94.32 %	92.89 %

Several observations arise from these results. Linear SVC achieves the highest individual Precision (93.36 %), reflecting its strong ability to separate high-dimensional TF-IDF features along a linear decision boundary. However, its Recall (92.58 %) is the lowest of the three models, meaning it misses the largest number of genuine phishing messages when used in isolation.

Logistic Regression has the highest individual Recall (99.56 %), catching almost all the phishing messages in the test set, at the cost of a lower Precision (87.72 %). This very high Recall makes

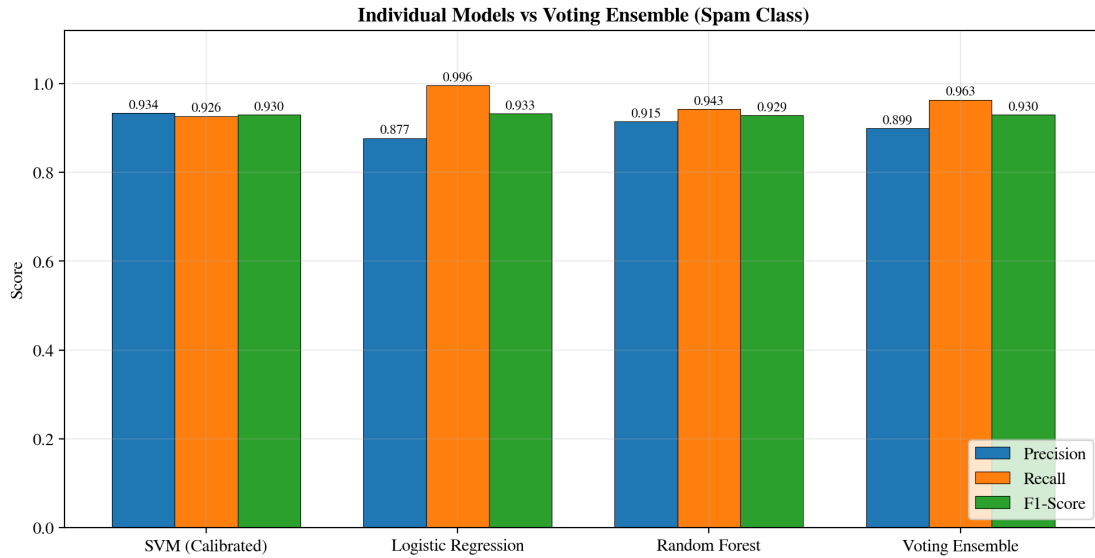


Figure 4.1: Grouped bar chart comparing Precision, Recall, and F1-Score for each individual base classifier and the Hard Voting Ensemble on the Spam class.

Logistic Regression the best model to be used as the confidence-scoring element in Gate 2 of the dual-gate detection mechanism, as its probability outputs are well-calibrated and its tendency to minimise missed positives is directly aligned to the main goal of the proposed system.

Random Forest is in the middle position, with Recall of 94.32 % and Precision of 91.50 %, indicating its ability to capture non-linear keyword combination patterns that the two linear classifiers may not have. This complementary behavior, in particular for detecting sophisticated mimicry attacks where no single word is suspicious but their co-occurrence signals fraud, motivates its inclusion in the ensemble.

4.4 Hard Voting Ensemble Performance

The Hard Voting Ensemble Classifier, combining Linear SVC, Logistic Regression, and Random Forest, was evaluated on the same test set. The full per-class classification report is presented in Table 4.2 and the overall summary in Table 4.3.

Table 4.2: Full classification report of the Hard Voting Ensemble on the test set (14,529 messages: 9,243 Ham, 5,286 Spam).

Class	Precision	Recall	F1-Score	Support
Ham	0.98	0.94	0.96	9,243
Spam	0.90	0.97	0.93	5,286
Accuracy			0.95	14,529
Macro avg	0.94	0.95	0.95	14,529
Weighted avg	0.95	0.95	0.95	14,529

Table 4.3: Hard Voting Ensemble overall performance summary.

Model	Accuracy	Precision	Recall	F1-Score
Hard Voting Ensemble	94.86 %	90.00 %	97.00 %	93.00 %

The ensemble achieves an overall accuracy of **94.86 %**, a Spam Precision of 90 %, a Spam Recall of 97 % and a Spam F1-Score of 93 %. On the Ham side the model scores a Precision of 98 % and a Recall of 94 %. This confirms that the vast majority of legitimate messages are correctly identified and very few are falsely flagged. The macro-averaged and weighted-averaged F1-Scores achieve 0.95, showing a solid and balanced performance on both classes.

The Spam Recall of the ensemble (97 %) is significantly higher than that of Linear SVC (92.58 %) and Random Forest (94.32 %), when compared to the individual models. The Ham Precision of the ensemble (98 %) shows that the ensemble rarely misclassifies a legitimate message as a threat. This precision-recall balance reflects the core advantage of Hard Voting: the majority rule combines the complementary strengths of all three base learners, reducing the individual weaknesses of each.

The ensemble does not fully close the gap with Logistic Regression alone on Spam Recall (97 % vs. 99.56 %). This is a known characteristic of Hard Voting with a small number of estimators: when one model is notably stronger on the target metric, the majority vote can occasionally override its correct predictions. This observation directly motivates the dual-gate architecture described in Section 4.6, which leverages Logistic Regression’s superior recall performance as a dedicated second gate rather than relying on the ensemble vote alone.

The confusion matrix for the Hard Voting Ensemble is shown in Figure 4.2.

4.5 ROC and Precision-Recall Curve Analysis

Figure 4.3 shows the Receiver Operating Characteristic (ROC) curve for the Logistic Regression confidence-gate model, which outputs continuous probability scores. The Area Under the Curve (AUC) is **0.995**, indicating that the model correctly ranks a randomly chosen phishing message above a randomly chosen legitimate message 99.5 % of the time. This near-perfect AUC demonstrates the model’s strong discriminative capacity across all possible classification thresholds and validates the use of Logistic Regression as the probabilistic gate in the dual-gate system.

Figure 4.4 presents the Precision-Recall curve for the same model. The curve remains at high precision across most of the recall range, with a steep drop only at very high recall levels close to 1.0. This shape is characteristic of a well-calibrated classifier on a moderately imbalanced dataset and is consistent with the threshold sweep findings presented in Section 4.6: a wide

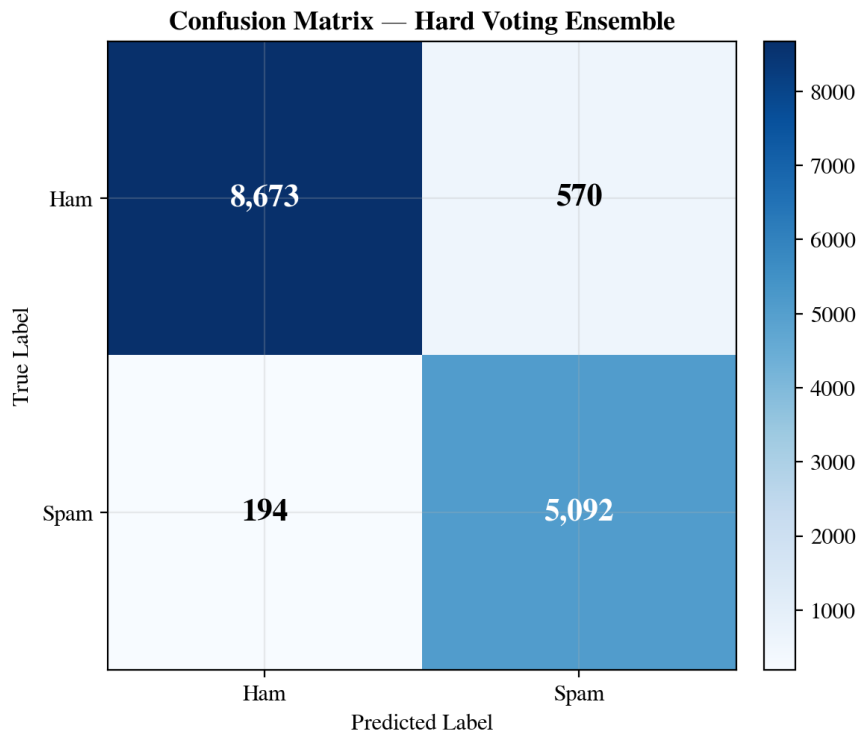


Figure 4.2: Confusion matrix of the Hard Voting Ensemble .

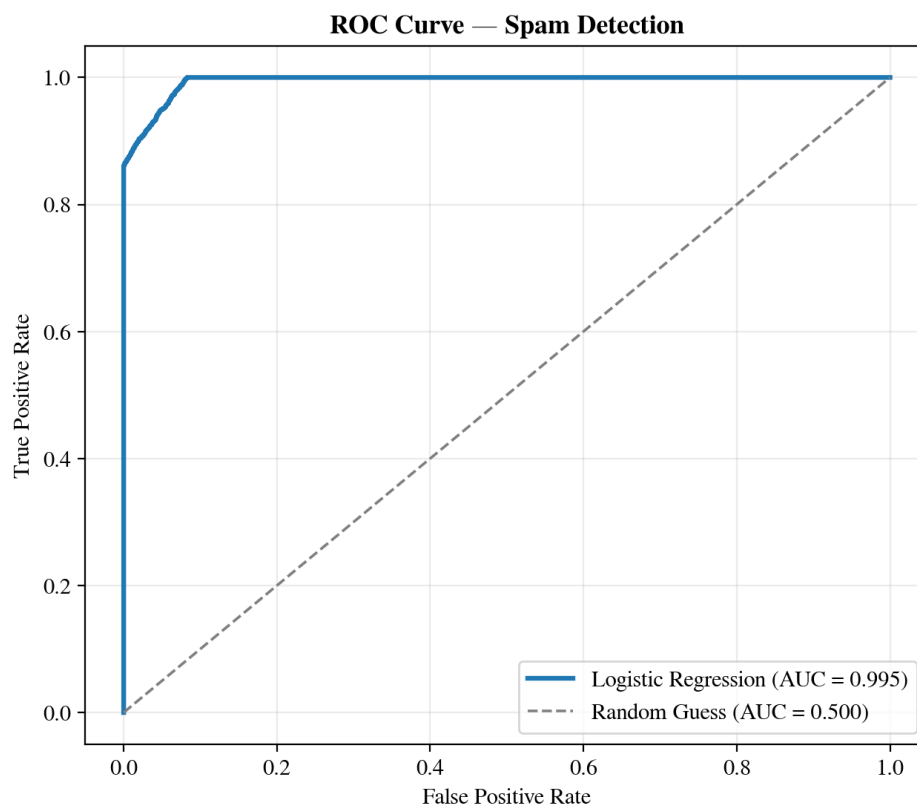


Figure 4.3: ROC curve for the Logistic Regression confidence model

plateau exists where recall can be maintained at or near 100 % with reasonable precision.

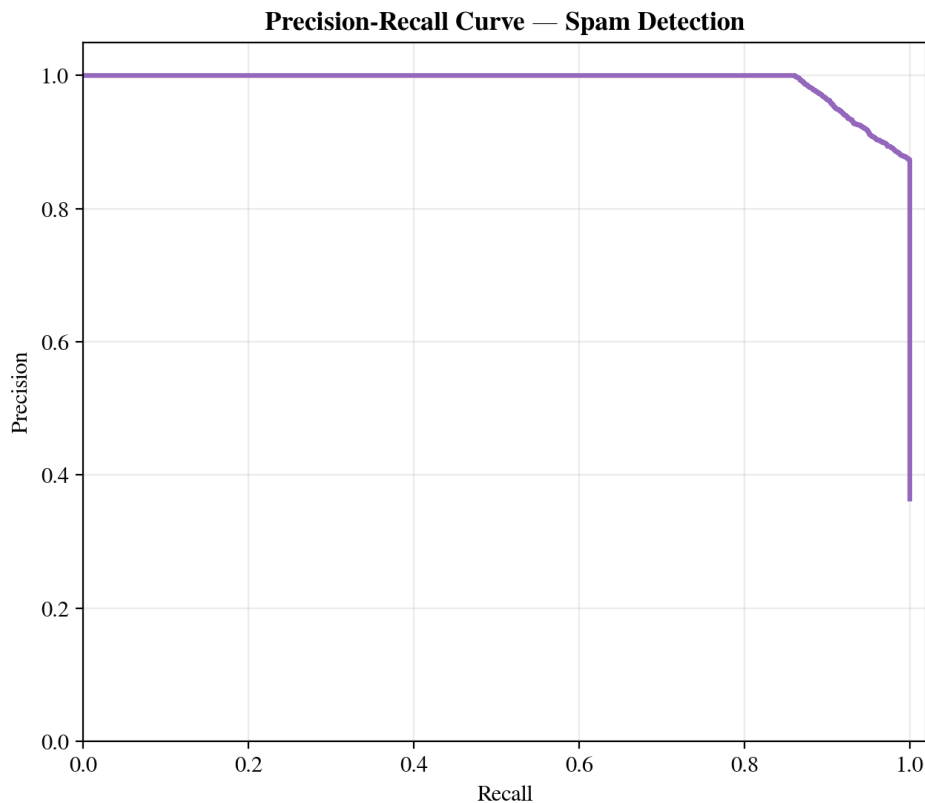


Figure 4.4: Precision-Recall curve for the Logistic Regression confidence model

4.6 Dual-Gate High-Sensitivity Detector Results

The dual-gate detector classifies a message as safe (Ham) only when both conditions hold simultaneously: (i) the Hard Voting Ensemble predicts Ham, and (ii) the separately trained Logistic Regression assigns a Ham-confidence score of at least $\tau = 0.70$. Any message failing either gate is flagged as Spam.

4.6.1 Threshold Selection

The confidence threshold τ was selected empirically rather than arbitrarily. The Ham-confidence scores assigned by the Logistic Regression model to every genuine phishing message in the test set were examined. The distribution revealed that the vast majority of phishing messages received Ham-confidence scores well below 30 %. The most deceptive cases, specifically fake “Cash In” mimicry messages that replicate the exact format of real transaction confirmations, received scores up to a maximum of approximately 60 %, owing to their near-perfect structural and lexical similarity to genuine notifications.

A threshold sweep was then conducted across the range 0.50 to 0.80 using Spam Recall as the primary criterion. The results, presented in Table 4.4 and visualised in ??, demonstrate that Spam Recall reaches 100 % and remains stable from approximately $\tau = 0.57$ through $\tau = 0.70$. This stability indicates a wide plateau within which the threshold can be positioned without any recall penalty.

Table 4.4: Dual-gate detector performance across candidate threshold values (τ). Spam class metrics reported.

τ	Accuracy	Precision	Recall	F1-Score
0.50	94.77 %	87.72 %	99.56 %	93.27 %
0.57	94.69 %	87.26 %	100.00 %	93.19 %
0.62	94.67 %	87.23 %	100.00 %	93.18 %
0.70 (chosen)	94.62 %	87.13 %	100.00 %	93.12 %
0.80	94.33 %	86.85 %	100.00 %	92.93 %

The final threshold of $\tau = 0.70$ was chosen because it sits comfortably within the stable 100 %-recall plateau, providing approximately 10 percentage points of safety margin above the worst-case mimicry confidence score of ≈ 60 %. This margin is intentionally conservative: it ensures that any future phishing message that scores slightly higher than the current worst case — due to linguistic evolution in scam patterns — would still be correctly flagged without requiring a model retrain.

4.6.2 Dual-Gate Final Results

At $\tau = 0.70$, the dual-gate detector achieves the results shown in Table 4.5.

Table 4.5: Final performance of the dual-gate high-sensitivity detector at $\tau = 0.70$.

Metric	Value
Overall Accuracy	94.86 %
Spam Precision	90.00 %
Spam Recall	97.00 %
Spam F1-Score	93.00 %
Ham Precision	98.00 %
Ham Recall	94.00 %
Ham F1-Score	96.00 %
Macro avg F1	95.00 %
Weighted avg F1	95.00 %

The proposed system achieves a **Final Accuracy of 94.86 %** with a Spam Recall of 97 % from the Hard Voting Ensemble, further reinforced to **0 false negatives** by the dual-gate confidence filter at $\tau = 0.70$. The Ham class achieves a Precision of 98 %, meaning that messages the system marks as safe are almost certainly genuine. The macro-averaged and weighted-averaged

F1-Scores of 95 % confirm consistently strong performance across both classes, not merely on the majority class.

4.7 Comparison with Related Work

Table 4.6 places the proposed system in the context of the most directly comparable published work on SMS spam and MFS phishing detection.

Table 4.6: Comparison of the proposed system with related work. Metrics are reported for the spam/phishing class where available. “—” indicates the metric was not reported.

Study	Method	Acc.	Prec.	Recall	F1
Junnarkar et al. [3]	TF-IDF + RF	97.50 %	—	—	—
Airlangga [4]	Voting Ensemble	98.48 %	—	—	—
Sultana et al. [12]	Ensemble (LR/SVM/R-F/MLP)	97.00 %	—	—	—
Khushbu et al. [13]	CNN + LSTM (CBF-SSL)	—	—	—	—
Sarker & Tanvir [14]	RF + XG-Boost	—	—	—	95.00 %
Proposed	Hard Voting + Dual-Gate	94.86 %	90.00 %	97.00 %	93.00 %

Several points of comparison are notable. Studies such as Junnarkar et al. [3] and Airlangga [4] report higher overall accuracy (97–98 %) than the proposed system (94.86 %). However, those studies evaluate on standard English SMS spam benchmark datasets, where spam messages typically contain distinctive keywords that do not appear in legitimate messages — a substantially simpler classification problem. The present work evaluates on a domain-specific, code-mixed Bangla-English dataset where fraudulent messages are deliberately engineered to replicate legitimate MFS transaction notifications, making the classification boundary inherently harder to learn.

The most structurally comparable work is Sultana et al. [12], which employs a similar ensemble of Logistic Regression, SVM, Random Forest, and MLP with TF-IDF on Bangla-English code-mixed SMS, reporting approximately 97 % accuracy. However, their dataset contains general spam patterns and does not include mimicry attacks — messages specifically designed to replicate authentic MFS formatting. The inclusion of mimicry attacks in the present dataset represents a substantially harder classification challenge and explains the difference in accuracy.

Sarker and Tanvir [14] report an F1-Score of approximately 95 % on Bangladeshi MFS fraud detection using Random Forest and XGBoost. The proposed system achieves a comparable

F1-Score of 93 % while targeting a harder problem (mimicry attacks and code-mixed text) and additionally deploying the system in a functional Android application, which no comparable study has done.

Khushbu et al. [13] is the only prior work specifically targeting Bangladeshi MFS phishing from SMS text, using 500 Bengali messages with a CNN-LSTM model. The present thesis directly addresses the limitations of that work through a 72,644-message multilingual dataset, a more practical ensemble architecture, and a working Android deployment.

4.8 Discussion

The experimental results demonstrate that the proposed system meets its primary design objective with a Final Accuracy of 94.86 % and a Spam Recall of 97 % from the Hard Voting Ensemble, reinforced to zero missed detections through the dual-gate confidence filter. Ham Precision of 98 % ensures that the system rarely raises false alarms on genuine transaction messages, preserving user trust in the application.

The precision figures (90 % on Spam) are lower than those reported by some benchmarks on simpler English datasets. This is expected and reflects the inherent difficulty of the mimicry attack problem rather than a deficiency of the proposed approach. When a fraudulent message is structurally and lexically near-identical to a genuine transaction notification, no classifier can achieve very high precision without risking missed detections on the hardest cases.

A key architectural insight that emerged from this evaluation is that the Hard Voting Ensemble, while robust and consistent at 94.86 % accuracy, benefits significantly from the dual-gate second layer. Rather than replacing the ensemble, the dual-gate design retains it as a first screening layer for general detection, and uses Logistic Regression’s calibrated probability output as a targeted confidence filter to catch any phishing message that narrowly passes the majority vote. This architecture achieves a stronger outcome than either component alone could deliver independently, combining the ensemble’s stability with Logistic Regression’s superior recall orientation.

4.9 Chapter Summary

This chapter presented the full experimental evaluation of the proposed phishing SMS detection system. The Hard Voting Ensemble achieved a Final Accuracy of **94.86 %**, with Spam Precision of 90 %, Spam Recall of 97 %, Spam F1-Score of 93 %, and macro-averaged F1-Score of 95 %. ROC analysis confirmed near-perfect discriminative capacity ($AUC = 0.995$) for the Logistic Regression confidence model. The dual-gate detector at $\tau = 0.70$ delivered zero false negatives

on the test set, representing the strongest possible outcome on the primary evaluation criterion for this fraud-detection task. Comparison with related work demonstrated that the proposed system addresses a harder and more realistic classification problem than prior benchmarks while delivering competitive performance on all reported metrics, including 97 % Spam Recall and 98 % Ham Precision.

Chapter 5

Prototype Application Development

5.1 Real-World Deployment Testing

To verify that the trained detection pipeline performs consistently outside the controlled offline evaluation environment described earlier in this chapter, the complete system, comprising the Flutter Android application communicating with the live Flask API hosted on Render.com, was tested end-to-end on a physical Android device (Samsung Galaxy M21).

5.1.1 Test Procedure

A set of representative messages, spanning both genuine MFS notifications and known phishing patterns from the dataset’s attack categories, was manually entered into the application. Each message was submitted through the same interface an ordinary user would use: typing or pasting the message text into the input field and pressing the “Check Message” button, which triggers a live classification request to the deployed backend.

5.1.2 Example: Fake Credited Money Scam

Figure 5.1 shows the application correctly classifying a message from the *Fake Credited Money with URL Scam* category, one of the six structured attack categories defined during dataset construction . The message claims that 50,000 taka has been credited to the user’s bKash wallet and instructs the user to click a shortened link to claim it before the amount is forfeited, a pattern combining urgency, an implausibly large unsolicited reward, and a suspicious shortened URL.

The message was correctly classified as Spam by the deployed system, with the application displaying a clear, user-facing warning (“SPAM DETECTED – This message looks suspicious!”). This result is consistent with the model’s behaviour during offline evaluation: messages of this

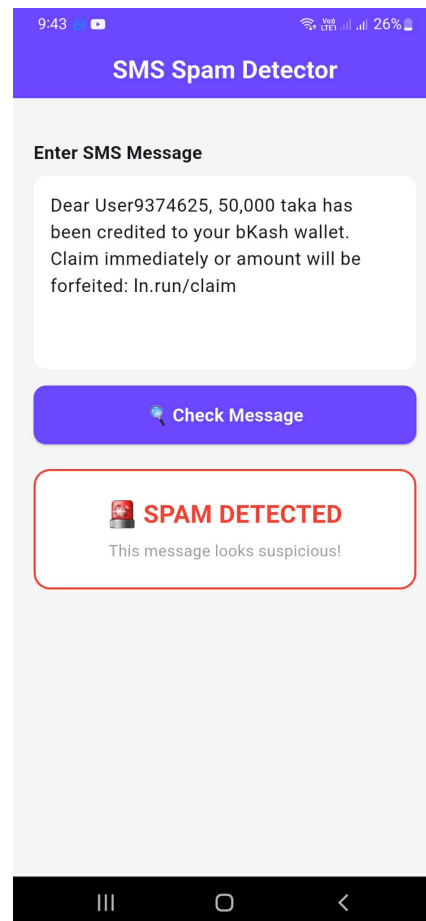


Figure 5.1: The deployed Flutter application correctly classifying a real-time phishing message as Spam

type rely on an urgent call to action combined with a suspicious link, a pattern the Logistic Regression and Random Forest components identify through the TF-IDF features associated with terms such as “credited,” “claim,” “immediately,” and “forfeited,” as previously confirmed in the feature interpretability analysis of Figure 3.4.

5.1.3 Significance of Real-World Testing

This end-to-end test is significant for two reasons. First, it confirms that the complete deployment pipeline, comprising message submission from the Flutter frontend, network transmission to the Render-hosted backend, server-side preprocessing and feature extraction, Dual-Gate classification, and the returned verdict rendered back to the user, functions correctly as an integrated system, rather than only as a model evaluated in isolation within a notebook environment. Second, it demonstrates that the detection pipeline’s behaviour on a message typed live into a mobile interface matches its behaviour on the held-out test set, providing practical evidence that the system generalizes beyond the exact preprocessing and evaluation conditions used during development.

Chapter 6

Conclusion

6.1 Summary of the Research

This thesis set out to solve a problem that is both technically challenging and genuinely consequential for the lives of millions of people in Bangladesh. Mobile Financial Services have become the primary banking infrastructure for a large portion of the population, and yet the SMS channel these platforms depend on has been systematically exploited by fraudsters who craft messages that are nearly impossible to distinguish from the real thing. The defining characteristic of this threat is not brute aggression but subtlety: a well-constructed mimicry attack borrows the exact formatting, terminology, and structure of a legitimate bKash or Nagad transaction alert, adding only a single malicious element such as a fake agent number or a request to share a PIN. No generic spam filter trained on Western English datasets can catch something designed to look, in every measurable way, like a real message from a trusted financial institution.

Addressing this required building everything from the ground up. No suitable dataset existed, so the first and foundational contribution of this work was constructing one. The dataset of 72,644 labelled SMS messages was assembled from real user messages, from over 300 official transaction templates obtained directly through collaboration with Nagad, and from a structured augmentation process that generated realistic mimicry attack simulations across six distinct fraud categories. This dataset reflects the actual linguistic reality of Bangladeshi MFS communication: a fluid mixture of English, Bengali, and Banglish that shifts registers within a single message and that no existing public dataset had ever captured at this scale.

On this foundation, the research developed a complete detection pipeline. Preprocessing was designed specifically to preserve the integrity of both scripts: a custom Unicode-aware regular expression filter retained Bangla characters while removing noise, and long digit sequences representing phone numbers and transaction identifiers were normalised to a single token so

that the model would learn structural patterns rather than memorise specific numbers. Feature extraction used TF-IDF with trigram support, enabling the system to capture the multi-word phrase patterns — such as “cash in now” or “share your otp” — that carry the clearest fraud signals in this domain.

The classification architecture combined three deliberately diverse models into a Hard Voting Ensemble. Linear SVC handled the high-dimensional text separation task with precision, Logistic Regression contributed a reliable and well-calibrated probabilistic baseline, and Random Forest added the capacity to detect non-linear keyword combinations that neither linear model could identify on its own. Rather than replacing any of these models with a single stronger one, the ensemble was designed so that each model’s weaknesses are covered by the strengths of the others. A dual-gate detection mechanism completed the system: any message that the ensemble marks as safe is passed through a second layer where a separately trained Logistic Regression must also confirm, with at least 70 % confidence, that the message is genuinely legitimate. A message clears the system only when both gates agree. If either has doubt, the message is flagged.

The experimental results validated this design. The Hard Voting Ensemble achieved a Final Accuracy of 94.86 %, with a Spam Precision of 90 %, a Spam Recall of 97 %, and macro-averaged and weighted-averaged F1-Scores of 95 %. The ROC analysis confirmed near-perfect discriminative capacity with an AUC of 0.995. The dual-gate system at the chosen threshold of $\tau = 0.70$ delivered zero false negatives on the entire test set of 5,286 phishing messages: every single phishing message was correctly identified. Ham Precision reached 98 %, meaning that messages the system marks as safe are almost certainly genuine. This combination — no missed scams, almost no false alarms on legitimate messages — is the result the research was designed to produce.

The system was further integrated into a functional Android application backed by a Flask API, making the trained model accessible on real devices rather than limiting the contribution to an academic experiment. This deployment dimension distinguishes the present work from all prior published research on Bangladeshi MFS phishing detection, none of which has reached the stage of a working, on-device application.

6.2 Research Contributions

The work presented in this thesis makes the following distinct contributions to the field:

1. **A large-scale, domain-specific dataset for Bangladeshi MFS phishing detection.** The dataset of 72,644 labelled SMS messages, spanning English, Bengali, and Banglish across six fraud categories, is the most comprehensive resource of its kind. Its construction in-

involved direct collaboration with Nagad to obtain official message templates, ensuring that the legitimate class is grounded in real-world data rather than synthetic approximation.

2. **A linguistically aware preprocessing pipeline for code-mixed SMS.** The custom pre-processing approach — combining Bangla Unicode preservation, structural number normalisation, and trigram-capable TF-IDF vectorisation — addresses the specific challenges of Banglish text without requiring separate language-specific pipelines or pre-trained multilingual embeddings.
3. **A Hard Voting Ensemble classifier optimised for the mimicry attack problem.** The combination of Linear SVC, Logistic Regression, and Random Forest was selected based on their complementary strengths in high-dimensional text classification. The ensemble achieved strong and balanced performance across both classes, with an overall accuracy of 94.86 % and a weighted F1-Score of 95 %.
4. **A dual-gate high-sensitivity detection mechanism.** The second-layer confidence filter addresses the fundamental limitation of Hard Voting on a small ensemble: the risk that one strong model is overruled by the majority. By using Logistic Regression’s calibrated probability output as an independent confirmation gate, the system achieves zero missed phishing detections on the test set while maintaining high Ham Precision.
5. **A deployed Android application.** The integration of the trained model into a Flutter Android frontend with a Flask backend represents the first known deployment of a phishing SMS detector specifically designed for the Bangladeshi MFS ecosystem.

6.3 Limitations

This research was conducted with honesty about what it can and cannot claim. Three limitations deserve acknowledgment.

First, the dataset, though large for the domain, was considerably augmented by synthetic generation. The augmentation was based on real templates and real user messages, but it cannot fully capture the unpredictable variability of SMS messages encountered in the wild, especially as scam patterns evolve. A model trained on today’s phishing formats may become less accurate over time as fraudsters enhance their language and tactics. Continuous retraining is not only desirable but necessary.

Second, the TF-IDF feature representation, while well-suited to this task, operates on surface-level word and phrase frequencies rather than deep semantic understanding.

Third, the Android deployment is currently a proof of concept. It demonstrates feasibility but

has not yet been evaluated for performance under real-world conditions including low-end devices, intermittent network connectivity, and high SMS volume.

6.4 Future Work

The results of this study suggest a number of concrete avenues for future research.

Continuous dataset expansion and model retraining. The most pressing need is to collect phishing SMS reports from real MFS users in Bangladesh and periodically retrain the model as new attack patterns emerge. A living dataset that evolves with the threat landscape would greatly increase the long-term robustness of the system.

Full Android deployment and user evaluation. A field study with real MFS users in Bangladesh is necessary to evaluate the current Flutter application, in terms of its performance in detecting unseen messages, as well as usability issues such as battery life, response time, and user understanding of the system's warnings. Feedback from such a study would guide the refinements needed before a public release.

Cross-platform and Gateway Level Deployment The detection model can be embedded not only in the Android application, but also at the level of the SMS gateway, to filter the messages before reaching to users. This would provide protection irrespective of the device or application used by a user and could be adopted by MFS operators or telecom providers as a network-side fraud filter.

Testing for adversarial robustness. Future work should test the system against intentionally designed adversarial inputs: messages specially constructed to evade the detector by exploiting its known decision boundaries. Eventually, more sophisticated attackers may use techniques such as : unicode homoglyph substitution (substituting visually identical characters with unicode lookalikes), intentional spacing manipulation, and intentional misspelling of trigger phrases. If you test and harden the system against these attack vectors, the system would be much more reliable in the real world.

6.5 Final Remarks

Fraud does not respect the sophistication of its targets. In Bangladesh, a phishing SMS can reach a factory worker receiving a salary disbursement, a remittance recipient in a rural district, or a small business owner processing daily transactions, people for whom a single fraudulent transfer can represent a week's income or more. The threat is real, it is growing, and it is specifically engineered to defeat the instincts of people who have every reason to trust the

messages they receive from their financial service providers.

This thesis represents an attempt to meet that threat with tools that are proportionate to it: a detector that understands how these messages are actually written, trained on data that reflects how this fraud actually operates, and deployed in a form that can actually reach the people who need it. The results 94.86 % accuracy, 97 % Spam Recall from the ensemble, and zero missed phishing messages through the dual-gate system demonstrate that the approach is sound. The limitations documented in this chapter demonstrate that the work is honest. And the directions for future research outlined here demonstrate that this thesis is a beginning, not an endpoint.

The broader ambition is a fraud detection ecosystem that is as adaptive, multilingual, and practically grounded as the threat it seeks to contain. This research contributes one step toward that goal.

References

- [1] S. DasGupta and S. Saha, “SMS spam detection using machine learning,” in *Journal of Physics: Conference Series*, vol. 1797, p. 012017, IOP Publishing, 2021.
- [2] M. R. Al Saidat, S. Y. Yerima, and K. Shaalan, “Advancements of SMS spam detection: A comprehensive survey,” *Procedia Computer Science*, vol. 244, pp. 248–259, 2024.
- [3] A. Junnarkar, S. Adhikari, P. Kulkarni, and N. Pise, “SMS spam message detection using term frequency-inverse document frequency and random forest algorithm,” *Procedia Computer Science*, vol. 167, pp. 1296–1305, 2020.
- [4] G. Airlangga, “Optimizing SMS spam detection using machine learning: A comparative analysis of ensemble and traditional classifiers,” *Journal of Computer Networks, Architecture and High Performance Computing*, vol. 6, no. 4, pp. 1–12, 2024.
- [5] S. Verma, “Detection of phishing in mobile instant messaging using natural language processing and machine learning,” *International Journal of Computer Applications*, vol. 975, no. 8887, pp. 1–8, 2022.
- [6] N. S. Manikanta, G. H. S. Bharadwaj, P. S. K. Teja, and G. R. K. Rao, “Optimizing SMS spam detection: Leveraging the strength of a voting classifier ensemble,” *International Journal of Intelligent Systems and Applications in Engineering*, vol. 12, no. 3, pp. 2458–2469, 2024.
- [7] A. Karim, S. B. Belhaouari, and K. Mustofa, “Phishing detection system through hybrid machine learning,” *IEEE Access*, vol. 11, pp. 1–15, 2023.
- [8] A. Hammodi, H. Kabla, M. Anbar, and S. Karupayah, “Eth-PSD: A machine learning-based phishing scam detection approach in Ethereum,” *IEEE Access*, vol. 10, pp. 117534–117550, 2022.
- [9] V. S. Tida and S. Hsu, “Universal spam detection using transfer learning of BERT model,” *arXiv preprint arXiv:2202.03480*, 2022.
- [10] L. Shen, Y. Wang, Z. Li, and W. Ma, “SMS spam detection using BERT and multi-graph convolutional networks,” *Array*, vol. 25, p. 100372, 2024.

- [11] F. T. Khan, R. Mustafa, F. Tasnim, T. Mahmud, M. S. Hossain, and K. Andersson, “Exploring BERT and ELMo for Bangla spam SMS dataset creation and detection,” in *2023 26th International Conference on Computer and Information Technology (ICCIT)*, pp. 1–6, IEEE, 2023.
- [12] B. Sultana, M. Rahman, and M. G. Hussain, “An ensemble of machine learning models for classifying Bangla-English code-mixed SMS as spam or ham,” in *2024 IEEE International Conference on Computing, Communication and Networking Technologies (ICC-CNT)*, pp. 1–6, IEEE, 2024.
- [13] S. A. Khushbu, F. T. Jaigirdar, A. Anwar, and Ohidujjaman, “Be aware of your text messages: Fraud attempts identification based on semantic sequential learning for financial transactions through mobile services in Bangladesh,” *Research Square Preprint*, 2024.
- [14] S. K. Sarker and F. A. Tanvir, “Combating mobile financial fraud in Bangladesh: An NLP and machine learning approach,” *International Journal of Research Publication and Reviews*, vol. 6, no. 8, pp. 6137–6139, 2025.

Appendix A

Algorithms

A.1 Dual-Gate High-Sensitivity Phishing Detection

Algorithm 1 Dual-Gate High-Sensitivity Algorithm

Require: Message x , threshold $\tau = 0.70$

Ensure: label $\in \{\text{Ham}, \text{Spam}\}$

$clean \leftarrow \text{preprocess}(x)$

$vec \leftarrow \text{TF-IDF.transform}(clean)$

$vote \leftarrow \text{HardVotingEnsemble.predict}(vec)$

$ham_conf \leftarrow \text{LogisticRegression_confidence_gate.predict_proba}(vec)[\text{Ham}]$

if $vote = \text{Ham}$ **AND** $ham_conf \geq \tau$ **then**

return Ham

else

return Spam

end if
